EF234504 — Grafika Komputer

Pertemuan 4

CAMERA, PROJECTION & 3D

Lab. Grafika, Interaksi, Gim dan Analitik (GIGA)

Departemen Teknik Informatika

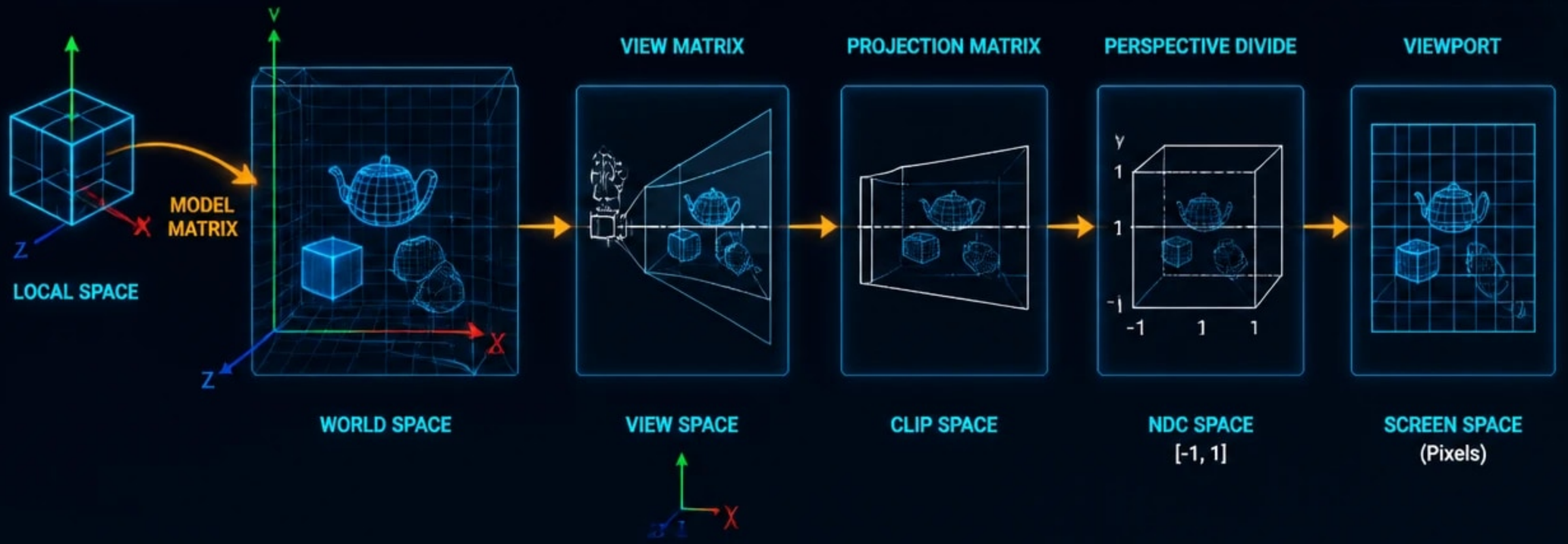
Posisi Materi

Pertemuan 3

Local
↓
Model Matrix
↓
World

Pertemuan 4

World
↓
View Matrix
↓
View
↓
Projection Matrix
↓
Clip
↓
Perspective Divide
↓
NDC
↓
Viewport
↓
Screen



Transformasi Berurutan
Dari objek lokal hingga layar

Ruang Koordinat
Setiap tahap memilki ruang sendiri

Matriks
Model, View, Projection mengubah koordinat

Hasil Akhir
Objek 3D muncul di layar 2D

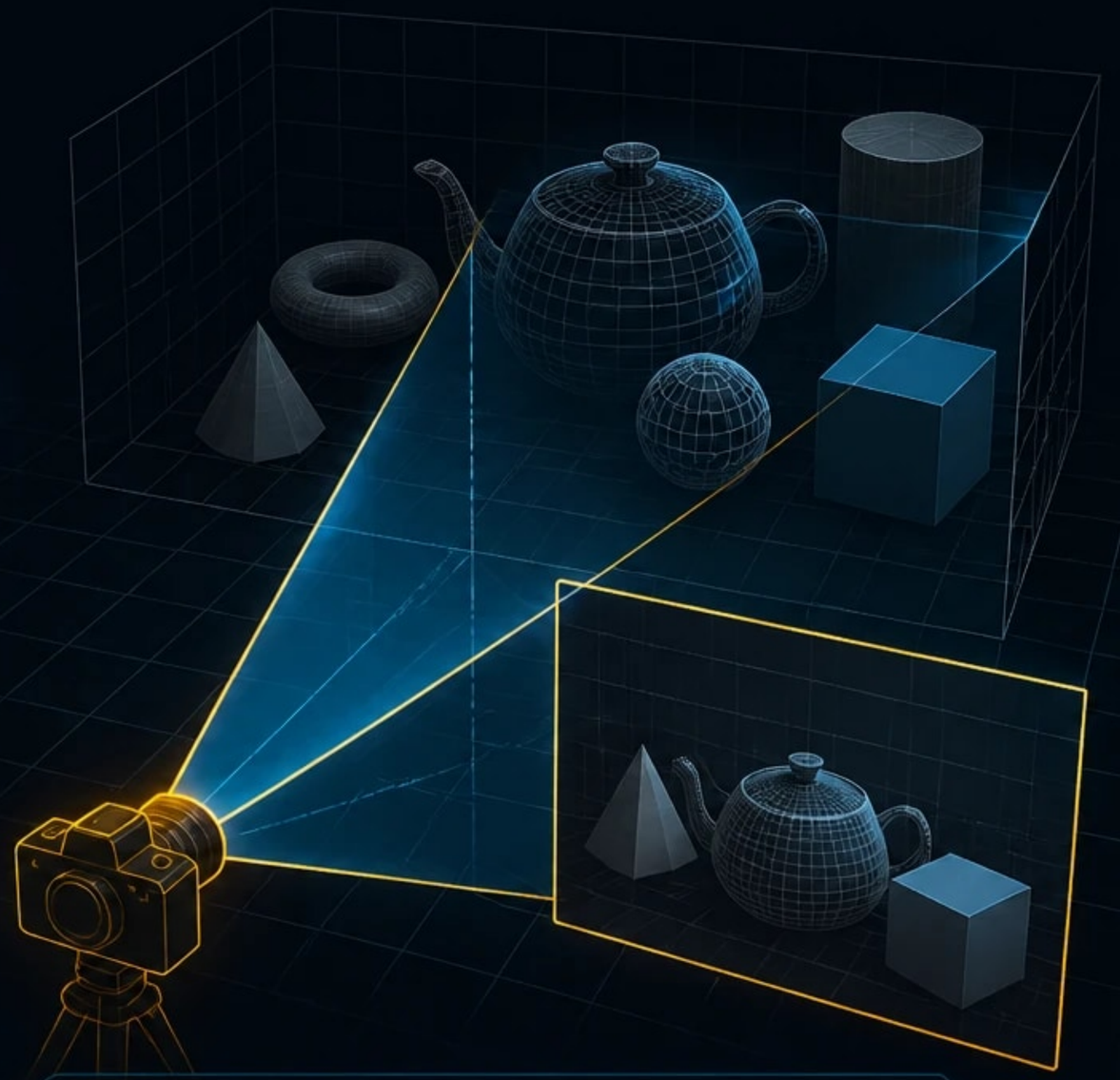
Mengapa Camera Dibutuhkan?

Scene 3D dapat berisi **banyak object.**

Pengguna hanya melihat: **bagian scene dari suatu sudut pandang tertentu.**

Camera menentukan:

- posisi pengamat
- arah pandang
- orientasi
- projection
- area scene yang terlihat



Intinya:
Camera memilih **apa yang dilihat**, **dari mana**, dan **bagaimana** cara melihatnya.

Camera Virtual

Camera pada grafika komputer adalah **representasi matematis.**

TUJUANNYA

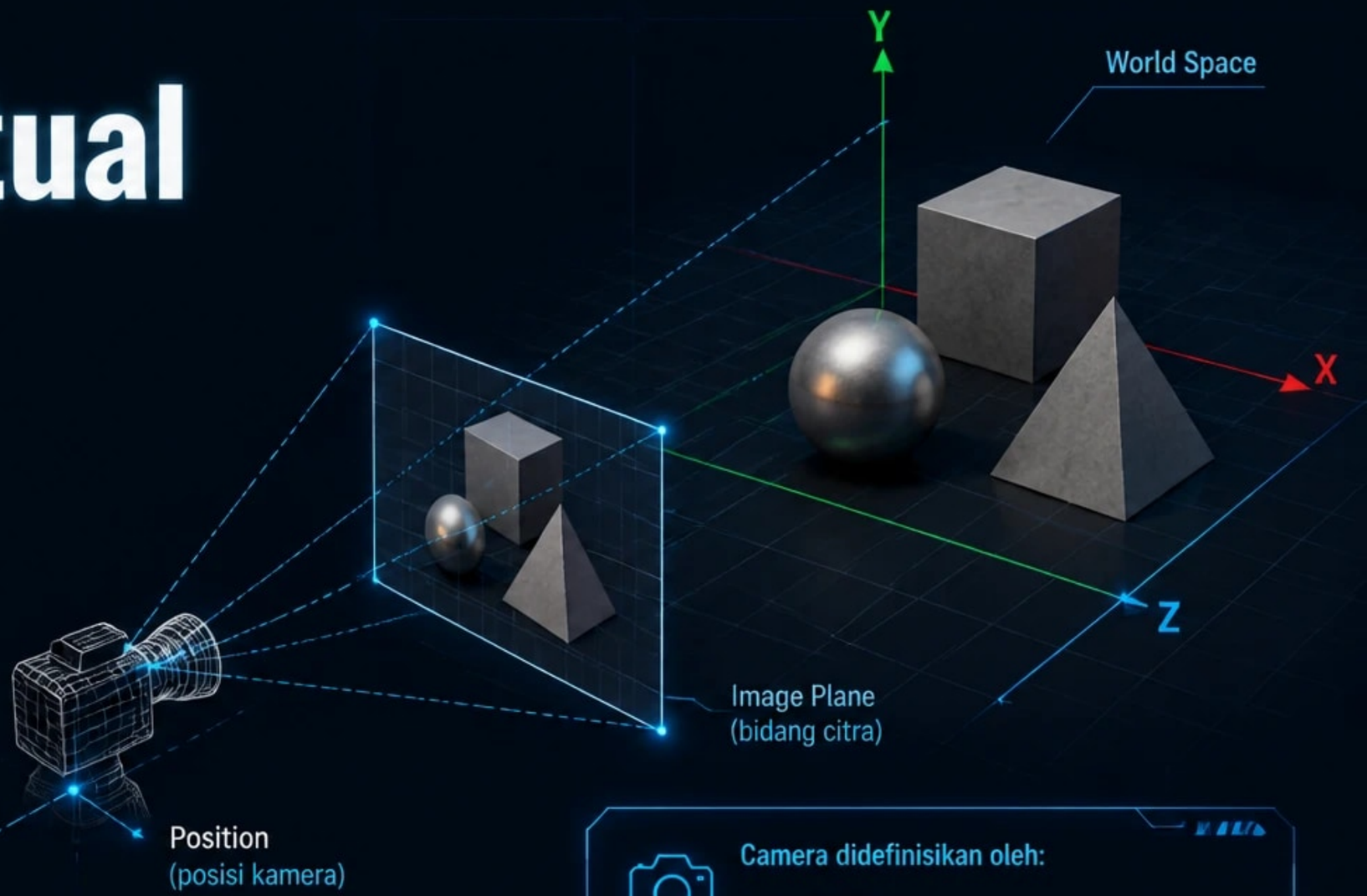
World

↓

Dilihat dari suatu posisi dan arah

↓

Image



PROSES UTAMANYA

VIEW TRANSFORMATION /view Matrix

Camera didefinisikan oleh:

- Posisi (eye)
- Arah pandang (target)
- Arah atas (up vector)
- Parameter proyeksi (FOV, aspect, near, far)

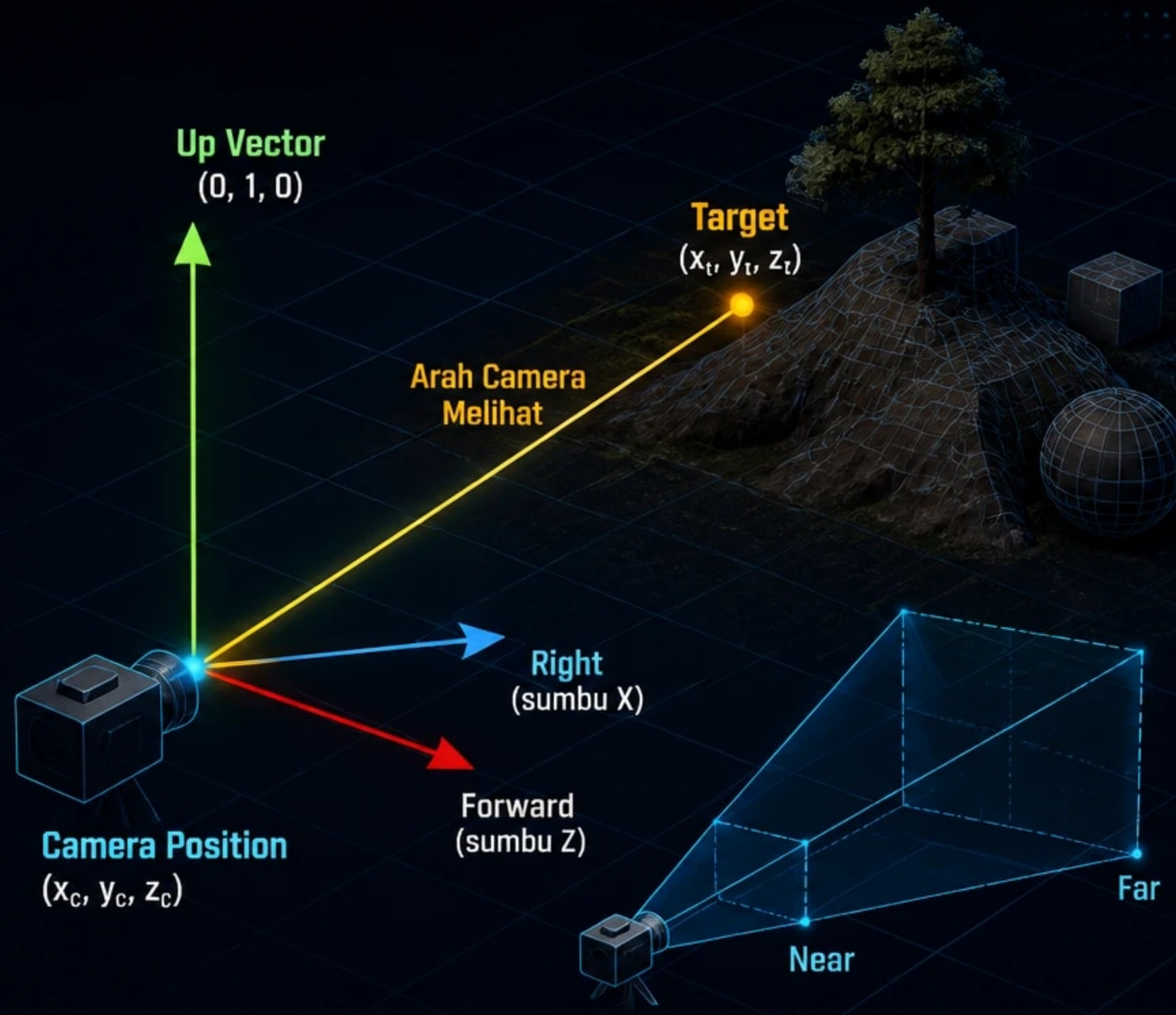
Parameter Dasar Camera

Camera sederhana dapat didefinisikan dengan:

- **Camera Position**
- **Target**
- **Up Vector**

Ketiganya menentukan:

lokasi camera	arah camera melihat	orientasi "atas"



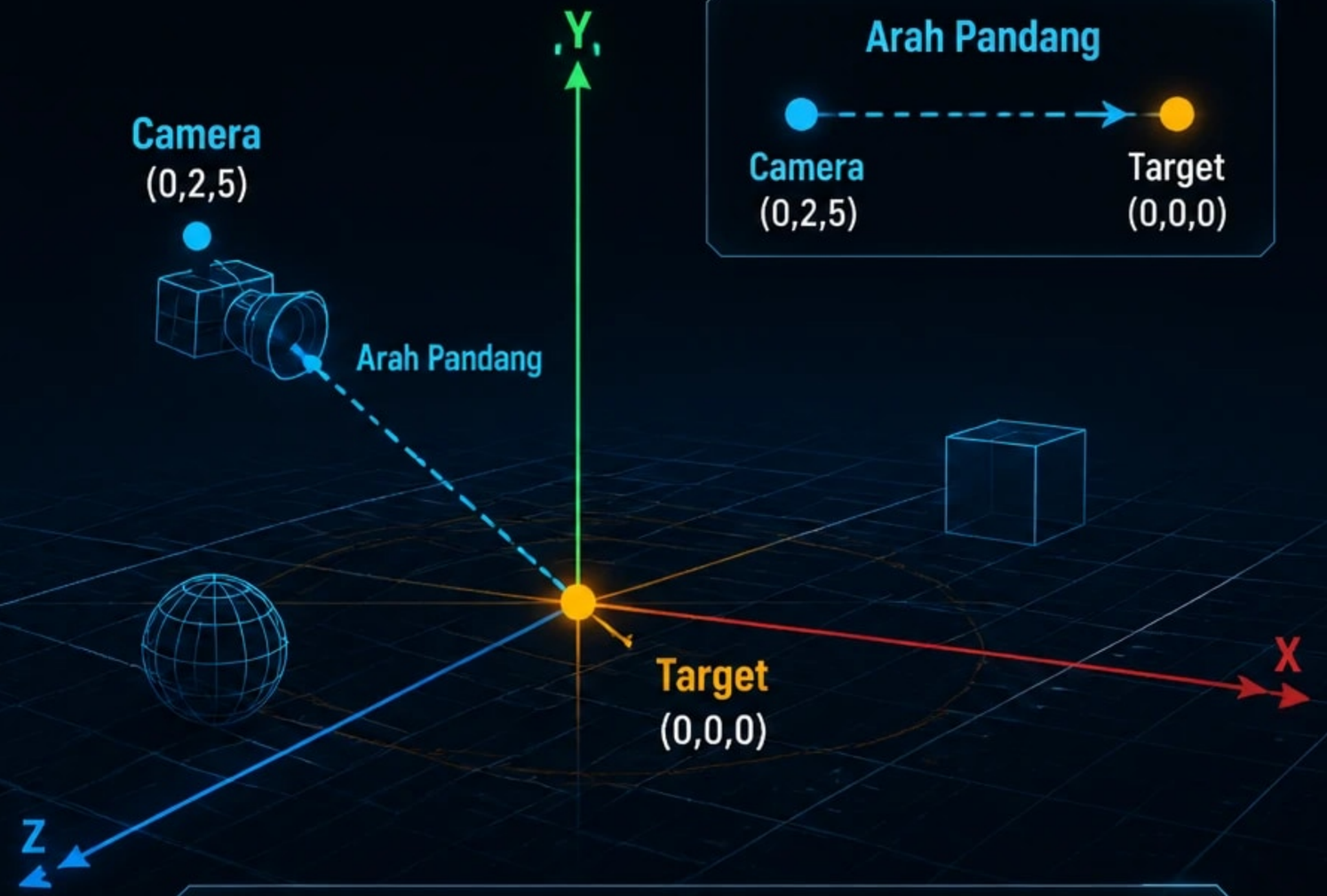
TARGET

Target adalah titik yang dilihat camera.

Camera = (0,2,5)
Target = (0,0,0)

Arah pandang berasal dari:

Target - Camera Position



Arah Pandang

Camera
(0,2,5)

Target
(0,0,0)

Arah Padang

Target - Camera Position

Note: arah pandang berupa vector (forward vector), nantinya akan berpengaruh pada tampilan object ketika terkena cahaya seperti shadow

Grafika Komputer

Forward Vector

Secara konsep:

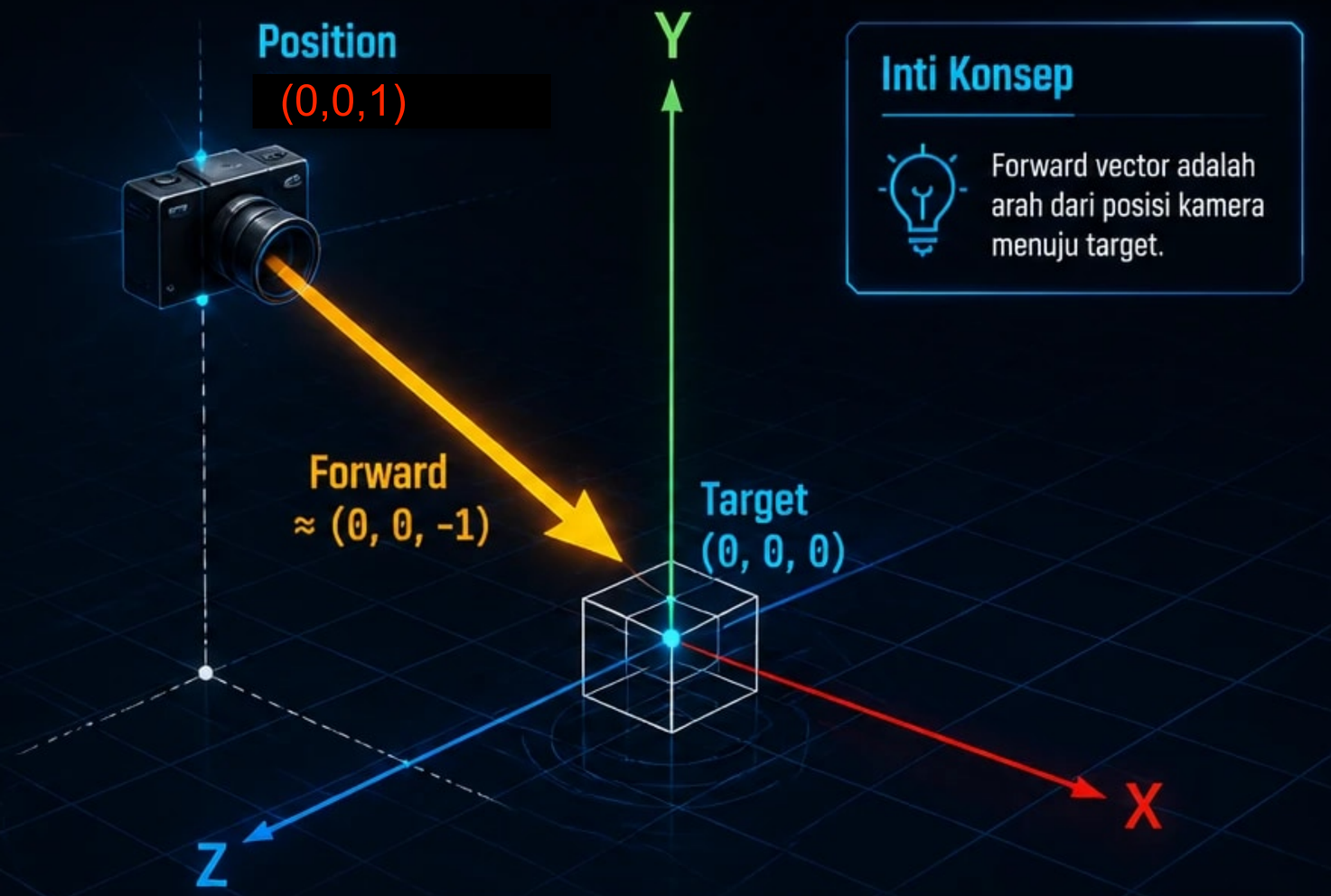
```
Forward =
normalize( Target - Position )
```

▷ Forward menunjukkan arah camera melihat.

Contoh:

```
Position = (0,0,1)
Target   = (0, 0, 0)

Forward  ≈ (0, 0, -1)
```



Inti Konsep

Forward vector adalah arah dari posisi kamera menuju target.

RUMUS UNTUK DIINGAT

```
Forward = normalize( Target - Position )
```

UP VECTOR

- Contoh umum:

  Up = (0,1,0)

- Artinya:

  +Y dianggap sebagai arah atas

- Up Vector diperlukan agar orientasi camera tidak ambigu.





Up Vector menetapkan "atas" dunia (world up) sehingga camera memiliki orientasi yang konsisten dan tidak ambigu.

Dengan Up Vector, kita mendefinisikan "atas" (biasanya +Y) sehingga kamera tidak bisa terbalik secara tidak sengaja.

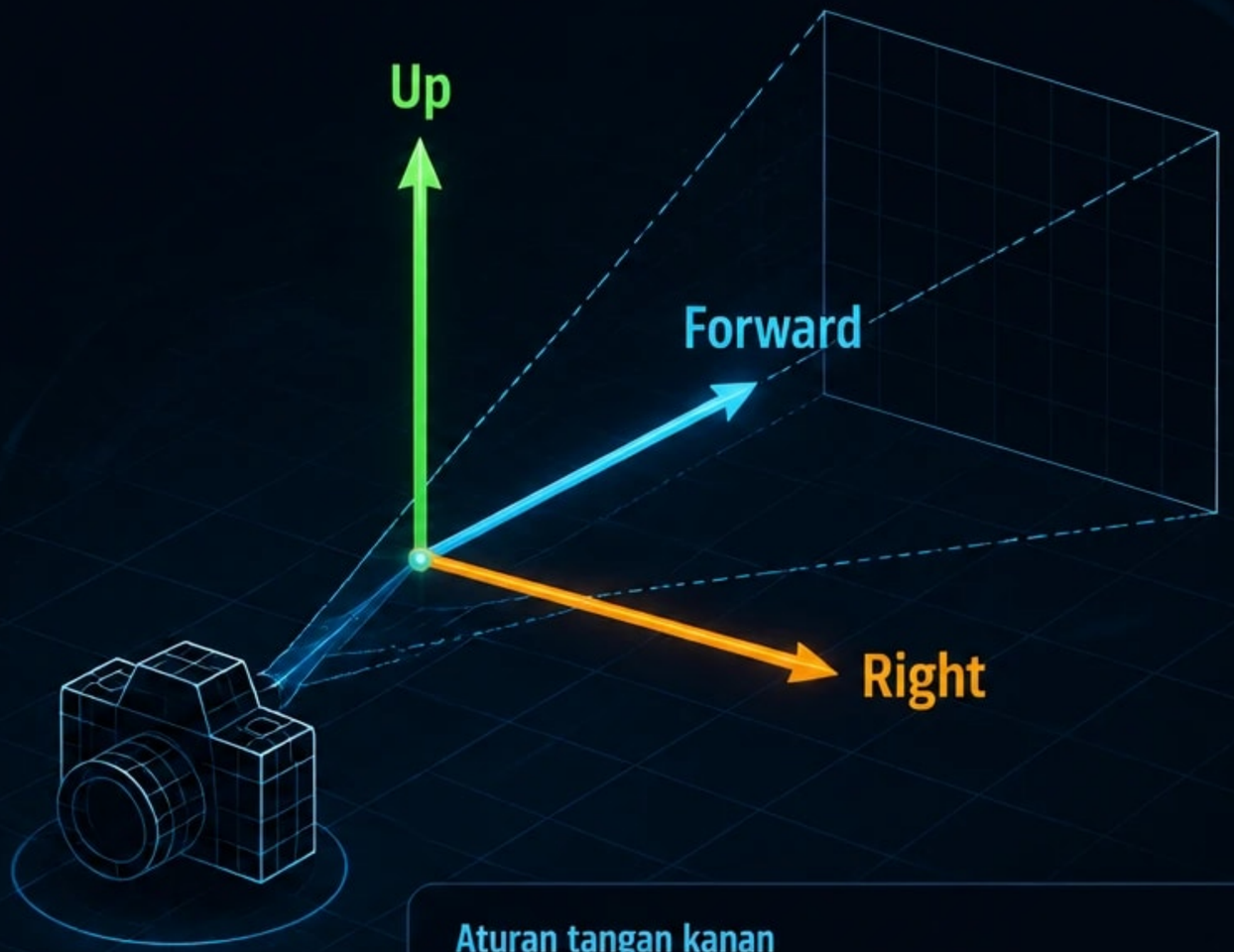
Right Vector

Basis camera menggunakan:
Forward, **Right**, **Up**

Right dapat dihitung
dengan **cross product**.

```
Right =
  normalize(
    Forward × Up)
)
```

Urutan cross product harus mengikuti **convention** yang digunakan.



Aturan tangan kanan



Jika jari telunjuk = **Forward**
dan jari tengah = **Up**
maka ibu jari = **Right**

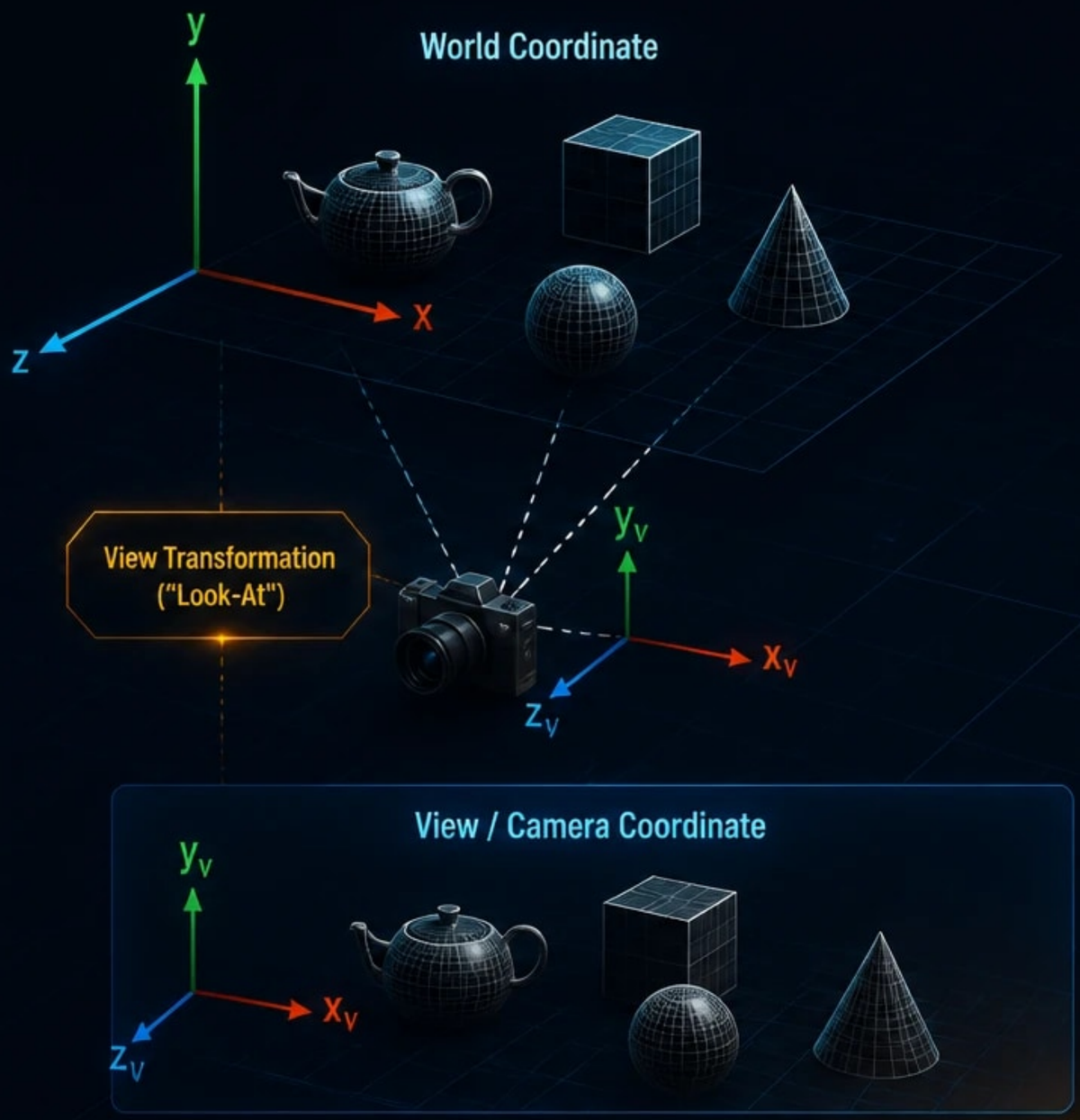
View Transformation

View Transformation mengubah:

World Coordinate → View / Camera Coordinate

Tujuannya:

menyatakan seluruh **scene** relatif terhadap **camera**.

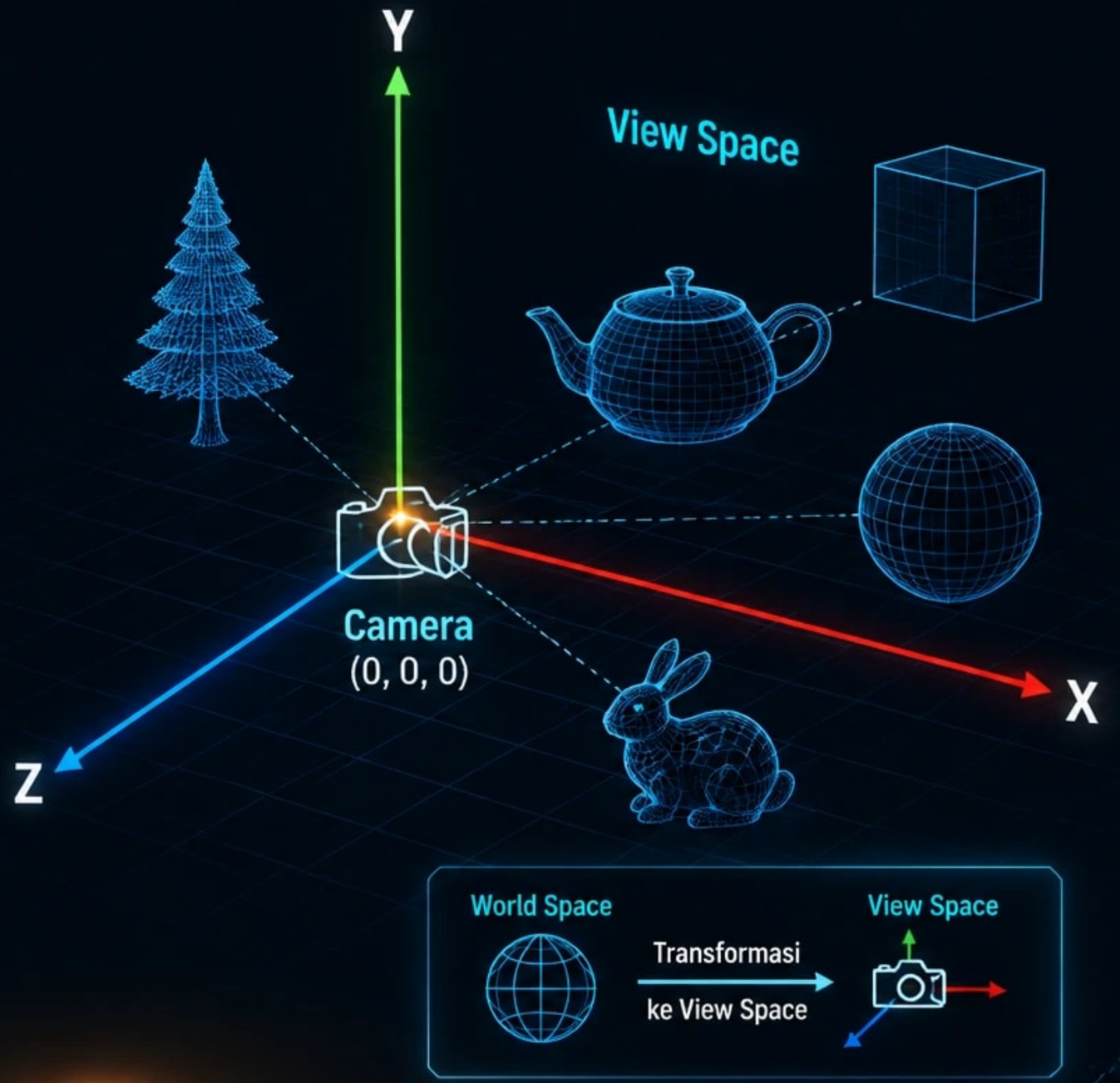


VIEW SPACE

Dalam View Space, **Camera** secara konseptual dianggap berada di **origin**.

Camera berada di **(0, 0, 0)** dengan orientasi standar.

World ditransformasikan relatif terhadap camera.



Analogi View Transformation

Jika **camera** bergerak ke kanan:

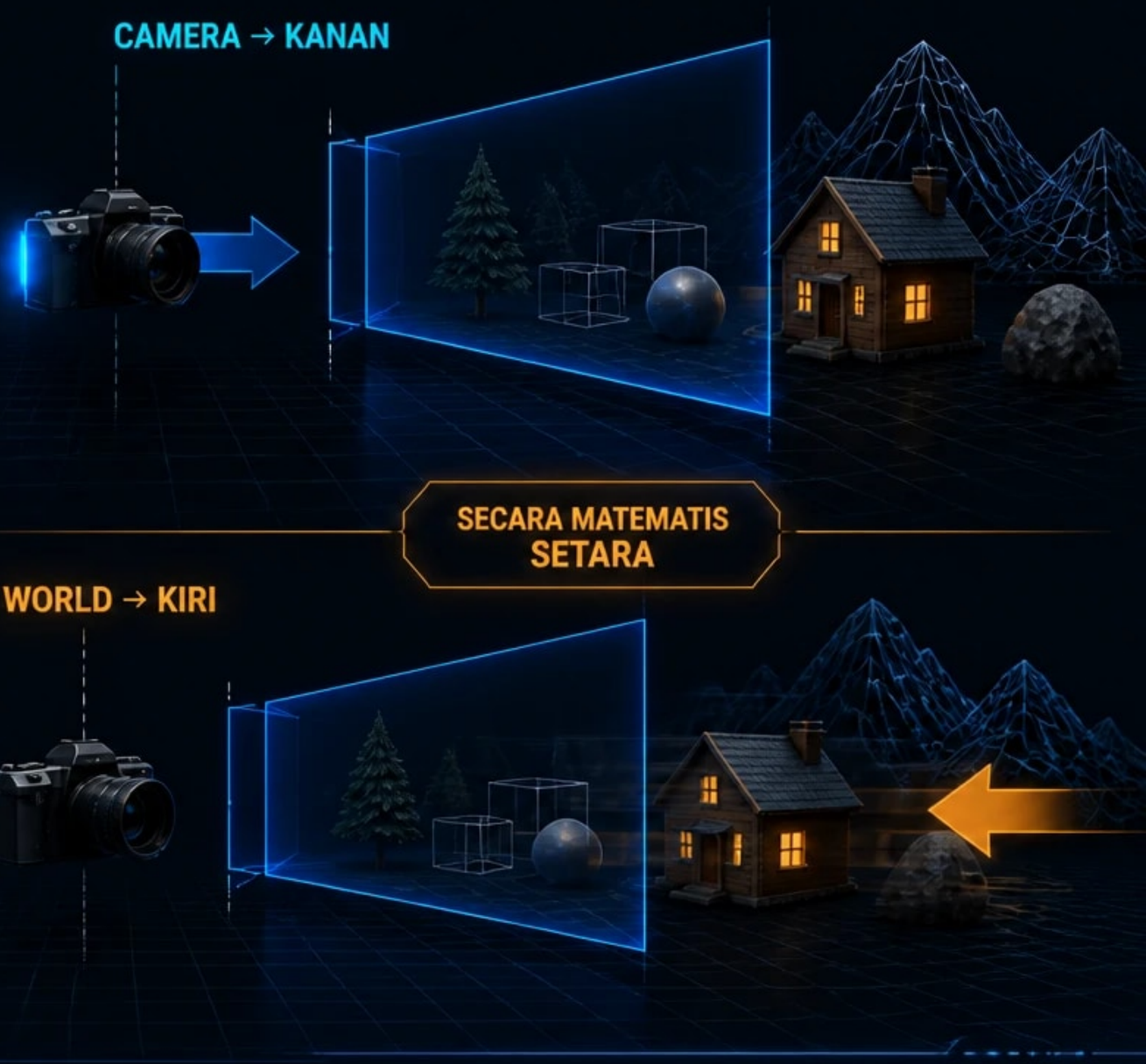
Camera → kanan

=

secara matematis setara dengan:

World → kiri

Karena yang dibutuhkan adalah posisi **world** relatif terhadap **camera**.



Grafika Komputer

View Matrix

View Matrix melakukan:

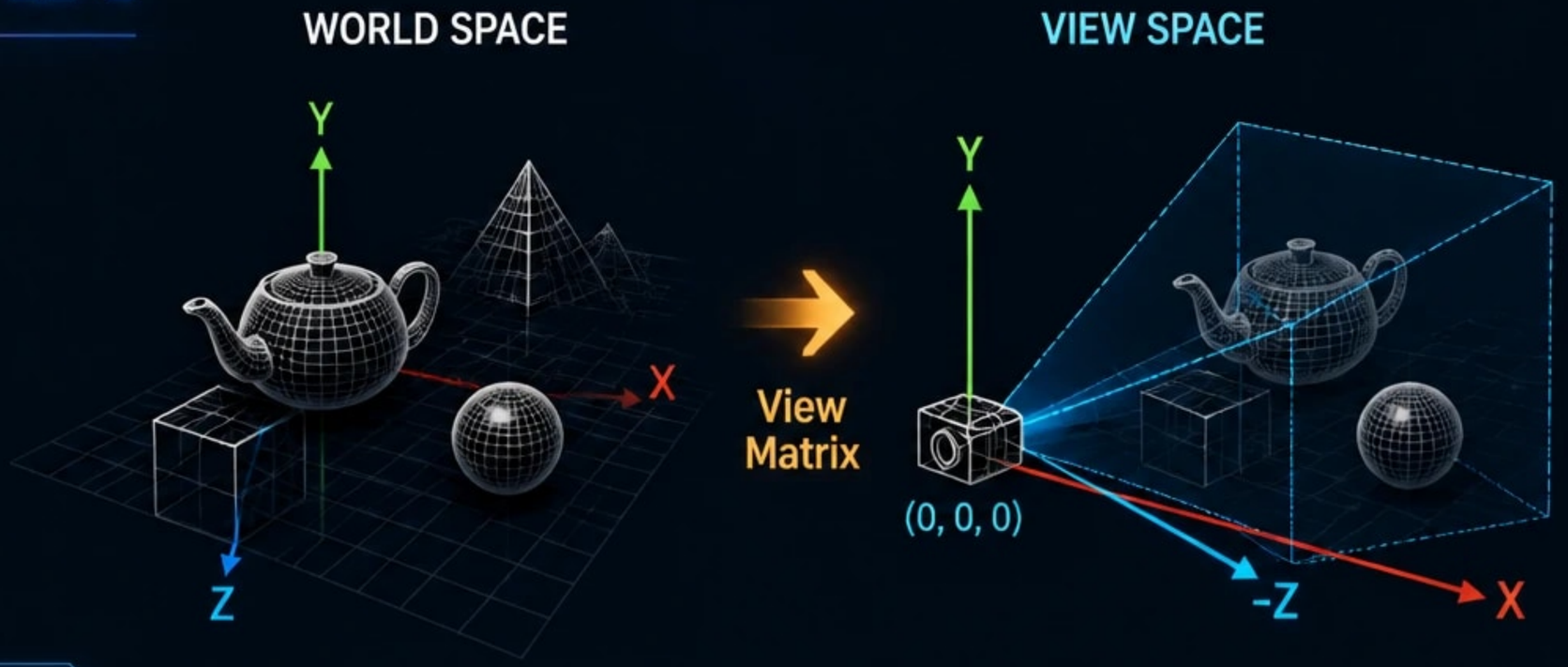
World Space

↓

View Space

Secara konsep:

$$viewPosition = ViewMatrix \times worldPosition$$



View Matrix = Inverse dari transformasi kamera

- Mentransformasikan dunia relatif terhadap kamera
- Kamera berada di origin (0,0,0) menghadap -Z

View Matrix dan Camera Transform



Camera mempunyai transformasi pada **World Space**.

View Matrix berkaitan dengan:

inverse

Secara sederhana:

View Matrix
≈ Inverse(Camera Transform)

Camera Transform (World Space)



$$C = \begin{bmatrix} R & T \\ 0 & 1 \end{bmatrix}$$

Posisi & orientasi camera di World Space

inverse

View Matrix

$$V = C^{-1} = \begin{bmatrix} R^T & -R^T T \\ 0 & 1 \end{bmatrix}$$

Mengubah dari World Space ke Camera Space (View Space)

View Matrix ≈ Inverse(Camera Transform)

Look-At Concept

Camera sering dibentuk dengan konsep:

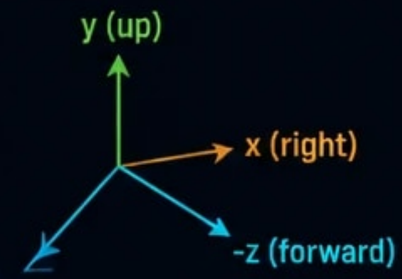
```
lookAt(
    position,
    target,
    up
)
```

Tujuan:
membentuk orientasi camera berdasarkan titik yang dilihat.

position	Posisi camera di dunia
target	Titik yang dilihat oleh camera
up	Arah "atas" untuk menentukan roll camera



CAMERA (VIEW) COORDINATE SYSTEM



Camera melihat sepanjang **-Z** dengan **Y** sebagai arah atas.

HASIL: VIEW DARI CAMERA



Pipeline Sampai View Space

Local Vertex
(Koordinat Lokal)

↓

Model Matrix
(Transformasi Model)

↓

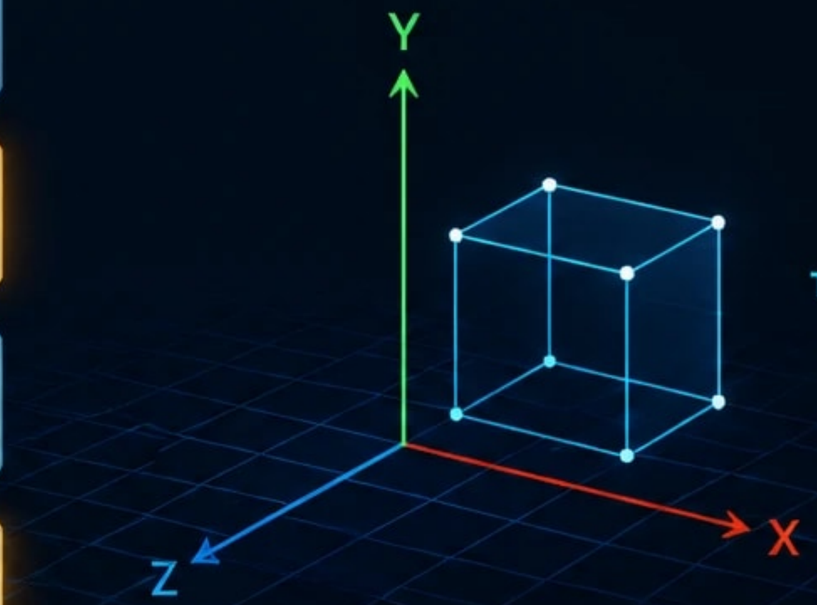
World Coordinate
(Koordinat Dunia)

↓

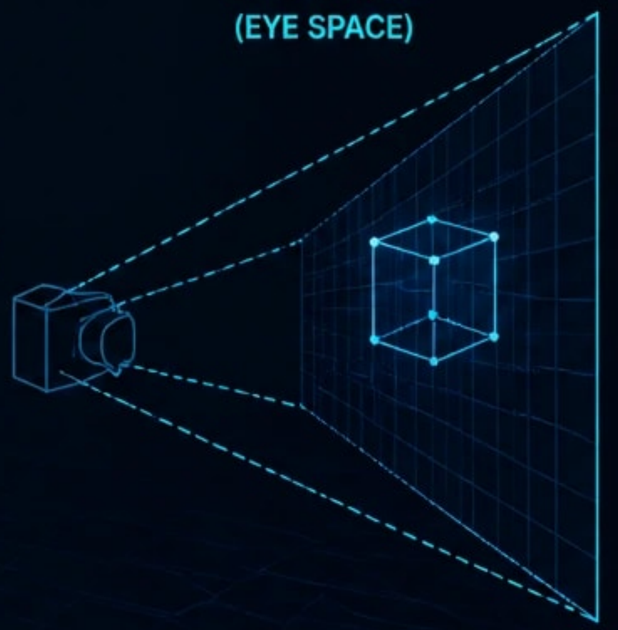
View Matrix
(Transformasi View)

↓

View Coordinate
(Koordinat View / Eye Space)



VIEW TRANSFORM



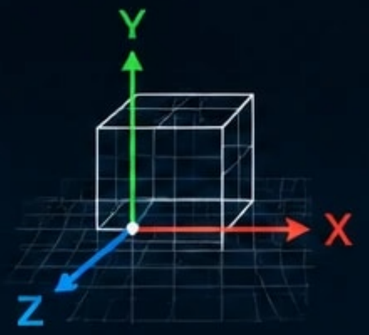
Setelah ini vertex siap diproyeksikan.

MODEL → WORLD → VIEW → PROJECTION → RASTERIZATION → FRAMEBUFFER

Masalah 3D ke Layar 2D

World
mempunyai:

X →
Y ↑
Z →





Layar
mempunyai:

X →
Y ↑



Kita memerlukan proses:

Projection

Apa Itu Projection?



Projection mengubah posisi **3D** menjadi bentuk yang dapat dipetakan ke layar **2D**.

Dua jenis utama:



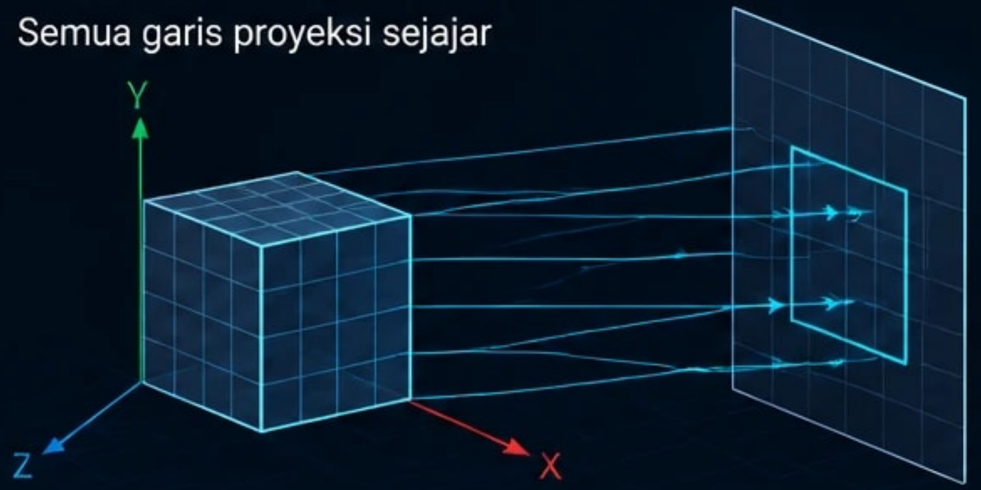
Orthographic Projection



Perspective Projection

ORTHOGRAPHIC PROJECTION

Semua garis proyeksi sejajar

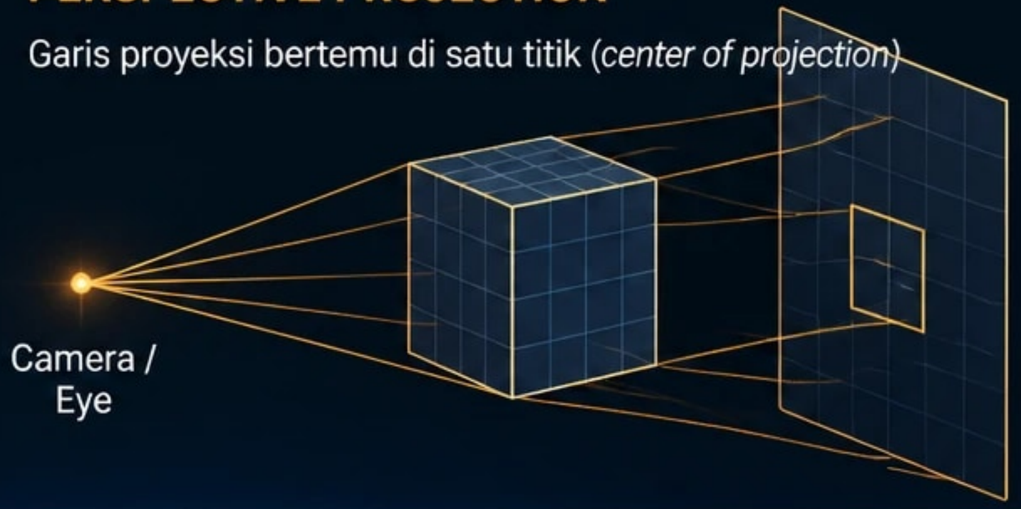


Hasil 2D



PERSPECTIVE PROJECTION

Garis proyeksi bertemu di satu titik (*center of projection*)



Hasil 2D



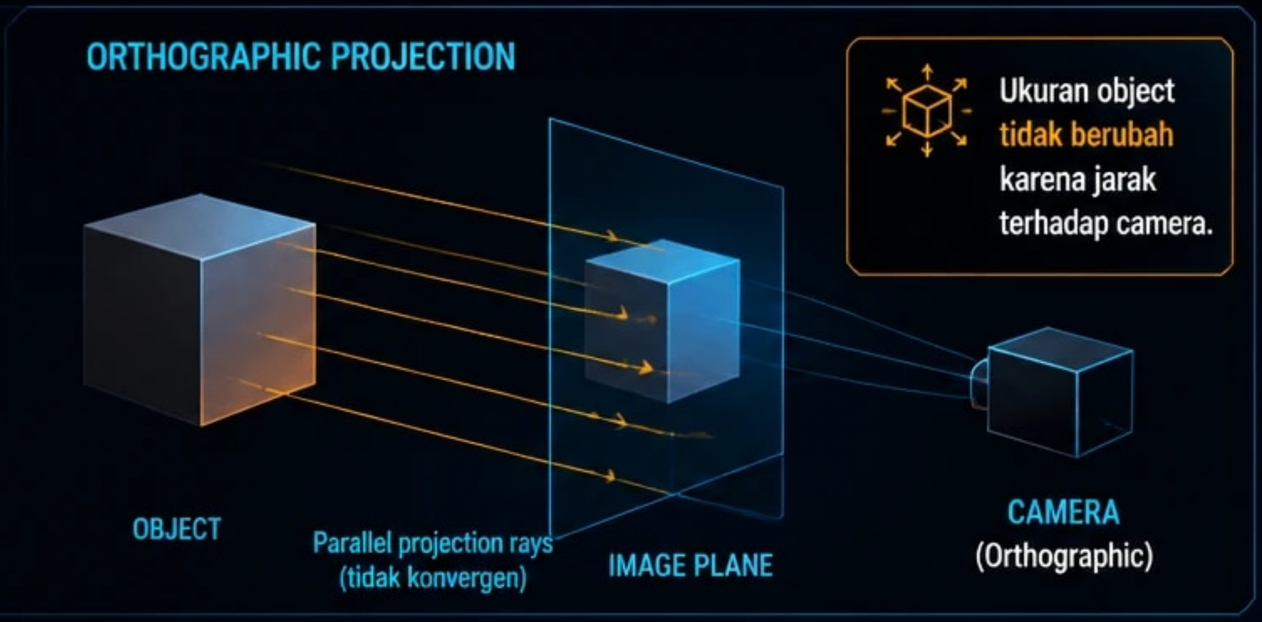
Orthographic Projection

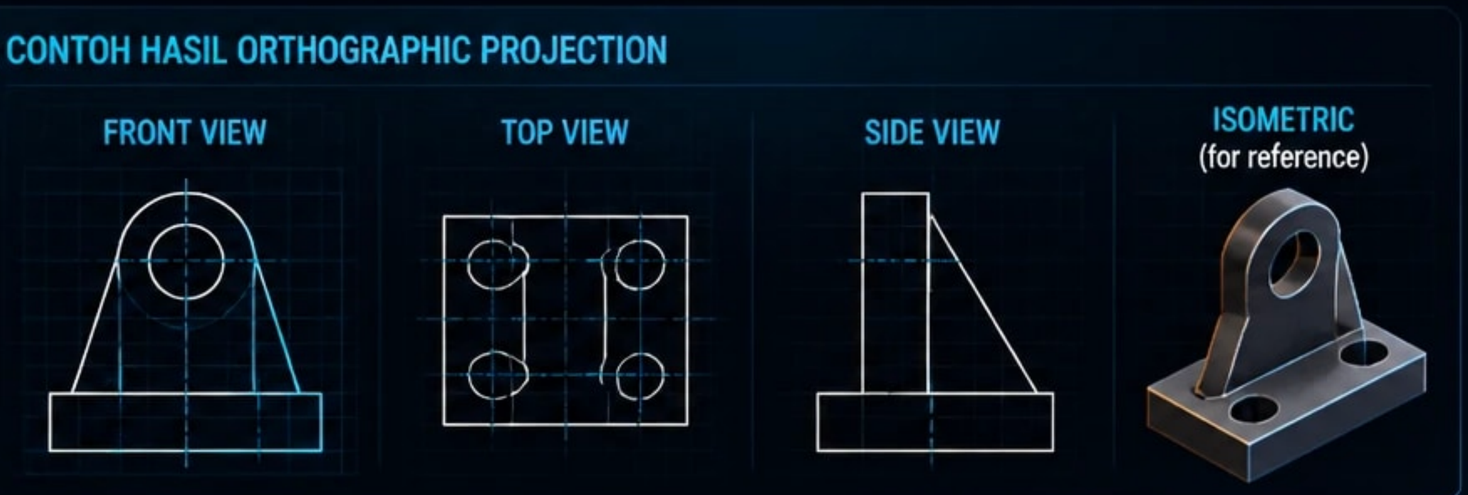
Ciri utama:

> ukuran object **tidak berubah** karena jarak terhadap camera.

Cocok untuk:

- CAD
- engineering drawing
- map
- visualisasi teknis
- editor 3D tertentu





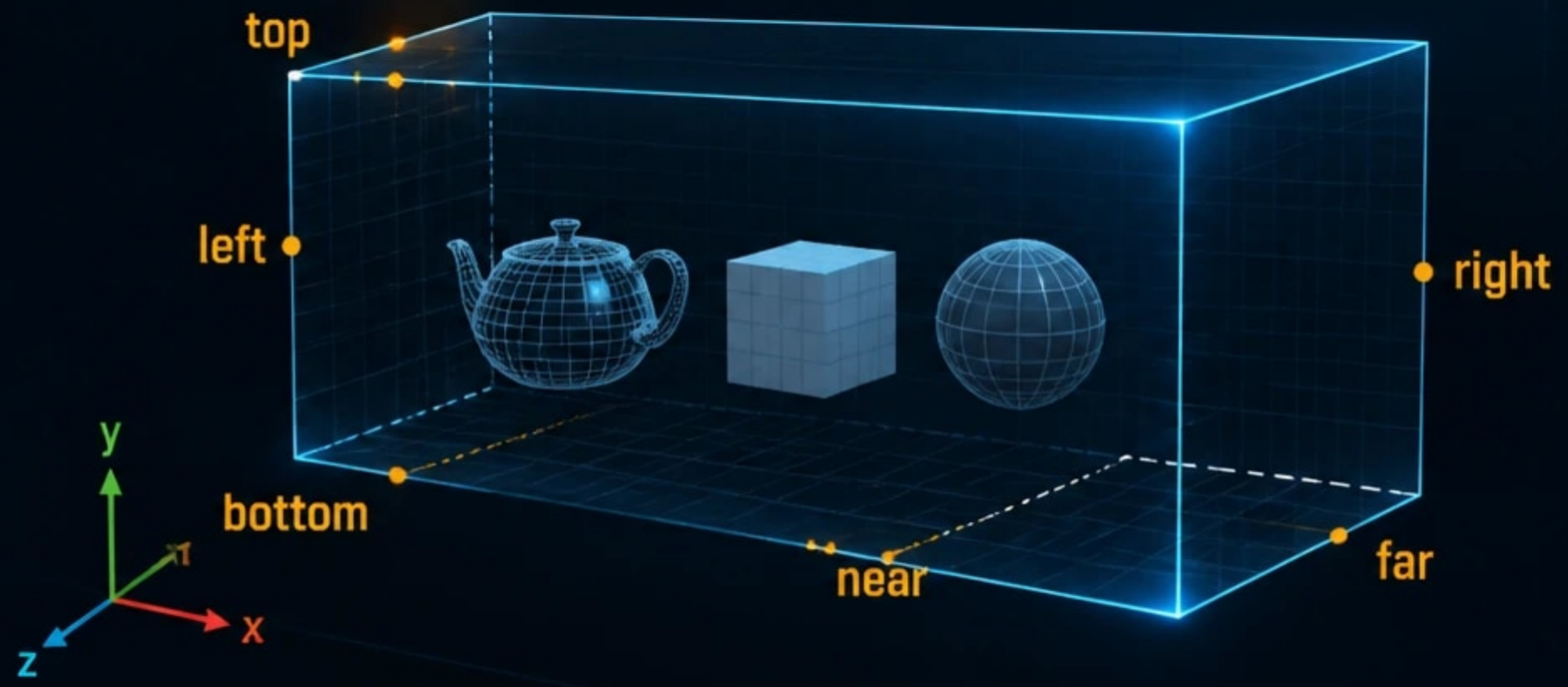
Sinar proyeksi **sejajar** → tidak ada perspektif → proporsi & ukuran **tetap konsisten.**

ORTHOGRAPHIC VOLUME

Orthographic menggunakan volume berbentuk **BOX**.

PARAMETER

- left
- right
- bottom
- top
- near
- far



Geometry di dalam volume dapat **diproyeksikan.**



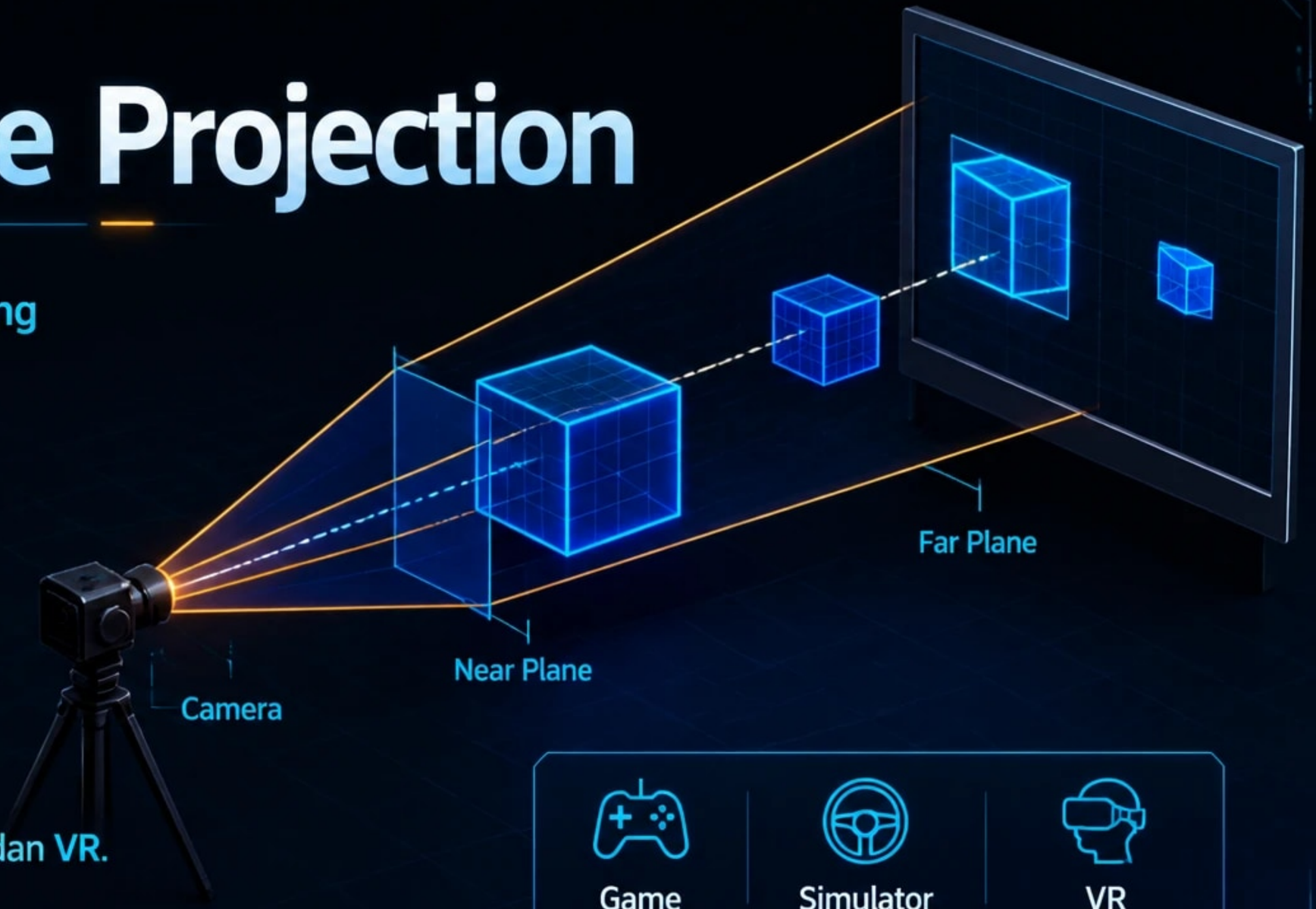
Perspective Projection

Perspective meniru **cara pandang** manusia/kamera **nyata**.

Object dekat
→ tampak lebih besar

Object jauh
→ tampak lebih kecil

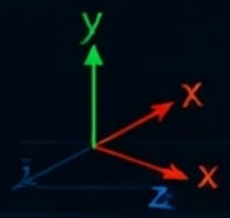
Umum pada **game**, **simulator**, dan **VR**.

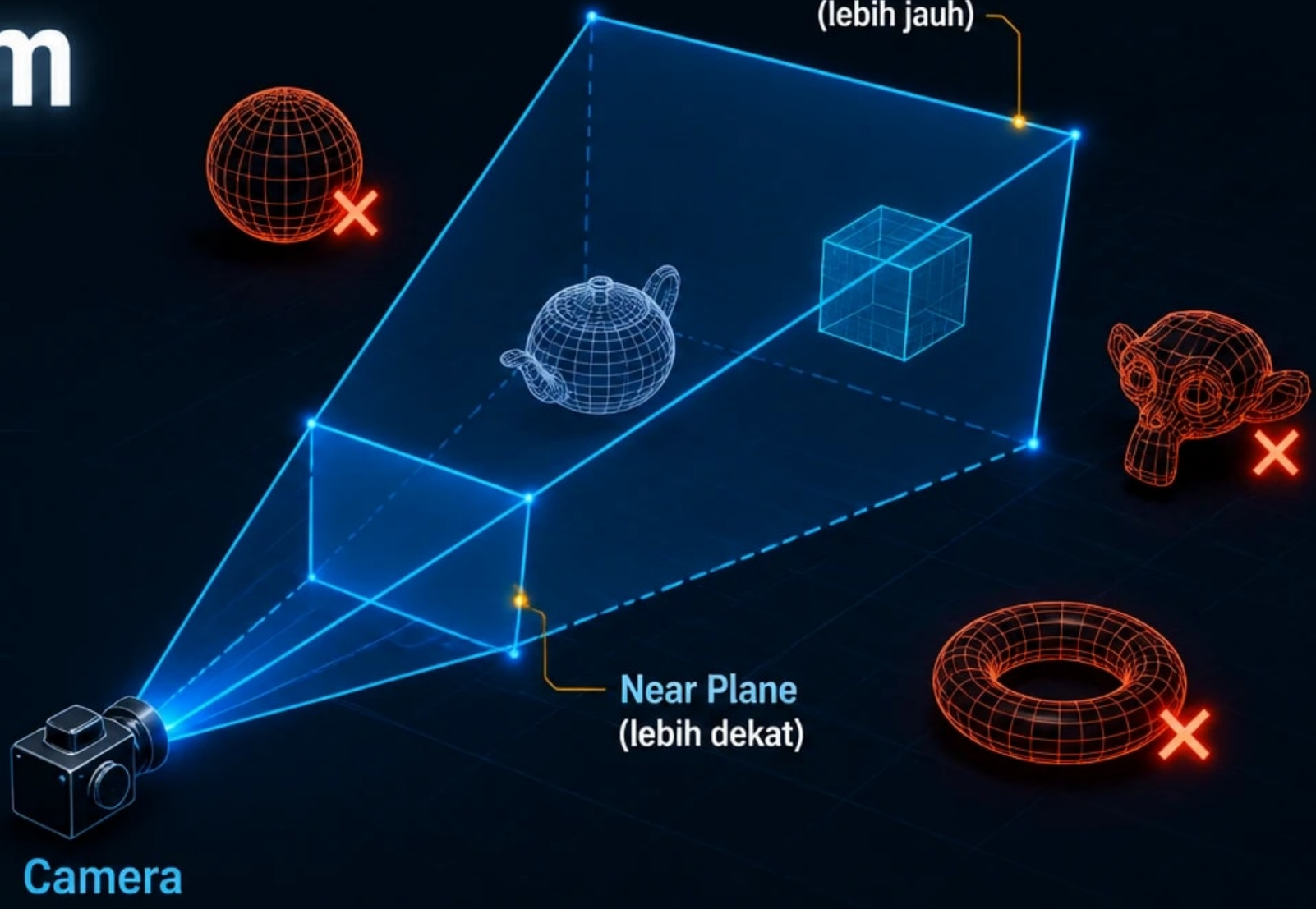


Game | Simulator | VR

View Frustum

- Perspective camera mengunakan volume berbentuk **frustum**.
- Dibatasi oleh **NEAR PLANE** dan **FAR PLANE**.
- Geometry di luar frustum dapat **di-clip**.





Field of View

FOV — Field of View

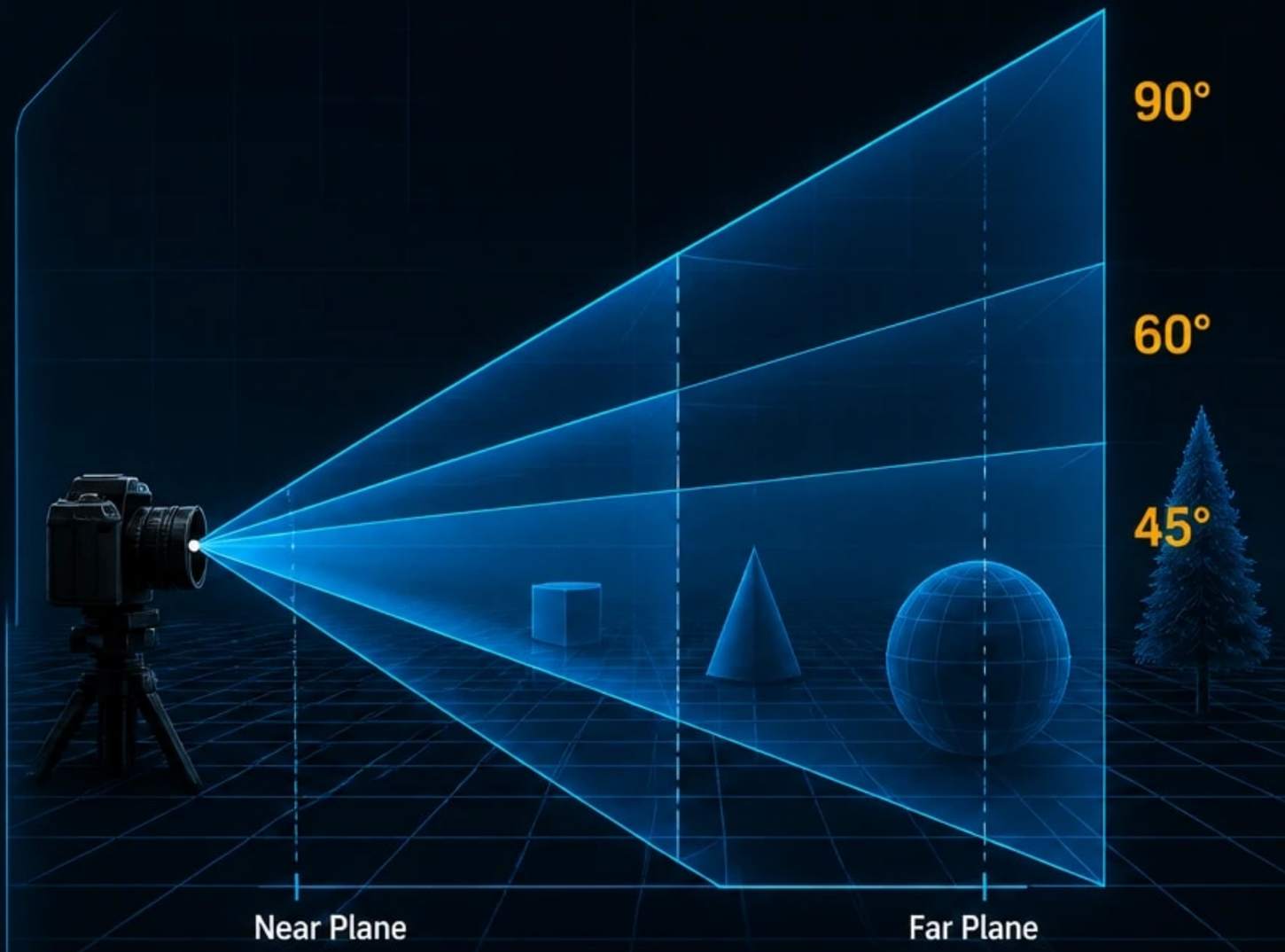
Menentukan luas
sudut pandang camera.

Contoh:



FOV kecil → lebih sempit / terasa zoom.

FOV besar → lebih luas.

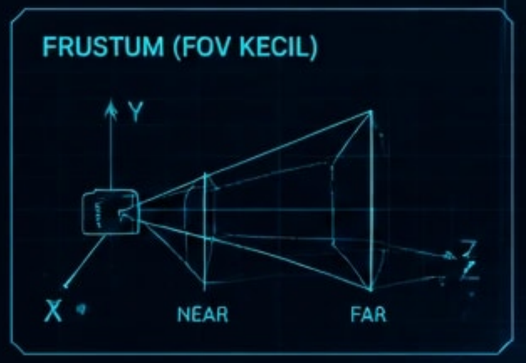


FOV mengatur seberapa banyak dunia yang terlihat di dalam layar kamera Anda.

FOV Kecil vs Besar

FOV KECIL

- area sempit
- object tampak besar
- kesan zoom
- distortion tepi lebih kecil

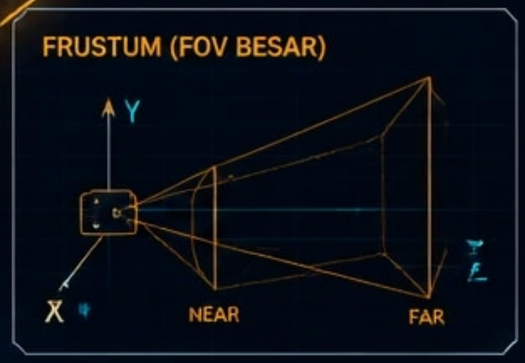






FOV BESAR

- area luas
- lebih dinamis
- distortion tepi lebih terlihat



FOV menentukan seberapa **luas dunia** yang terlihat melalui kamera.

Aspect Ratio

$$\text{aspect} = \frac{\text{width}}{\text{height}}$$

Contoh:

1920 / 1080

≈ 1.7778

= **16 : 9**

Aspect ratio menjaga proporsi **horizontal dan vertikal.**





Aspect Ratio yang Salah

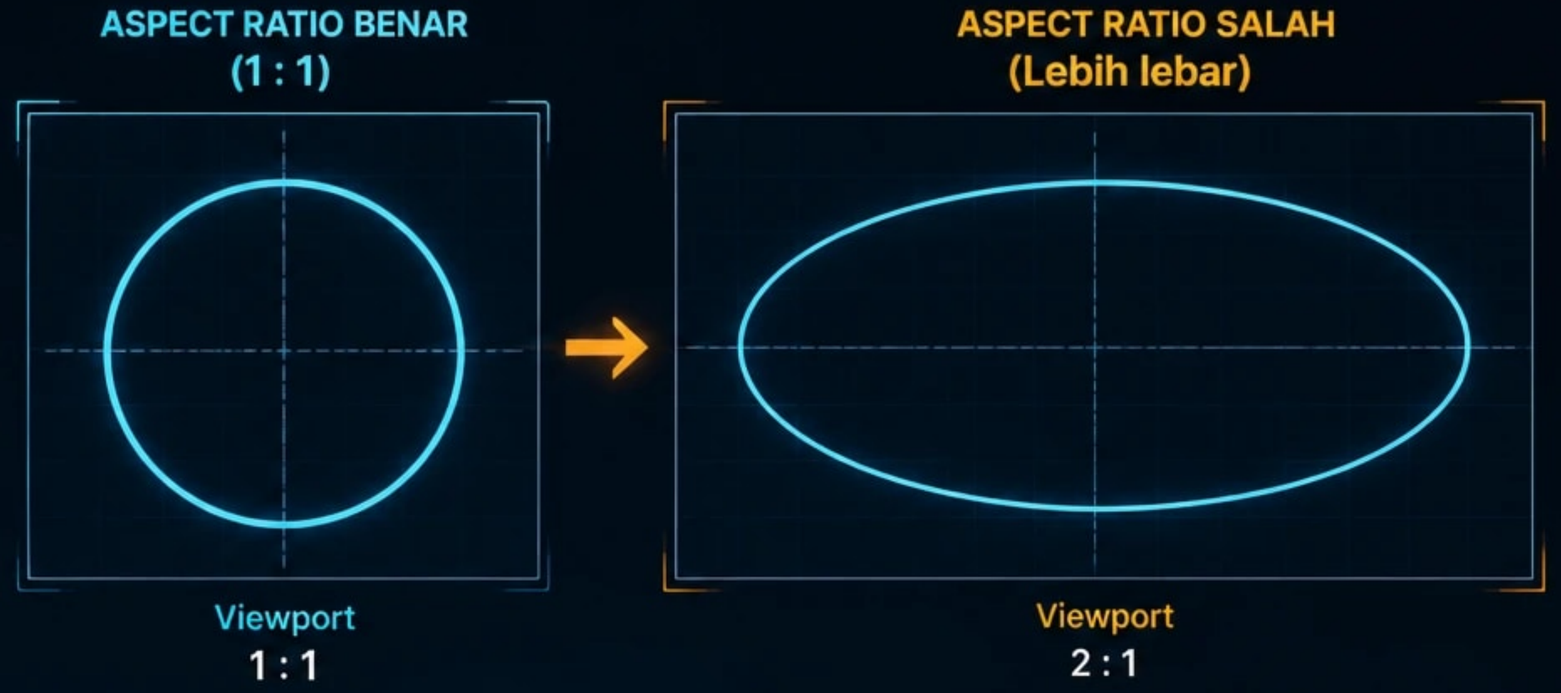
Jika aspect projection berbeda dengan **viewport**:

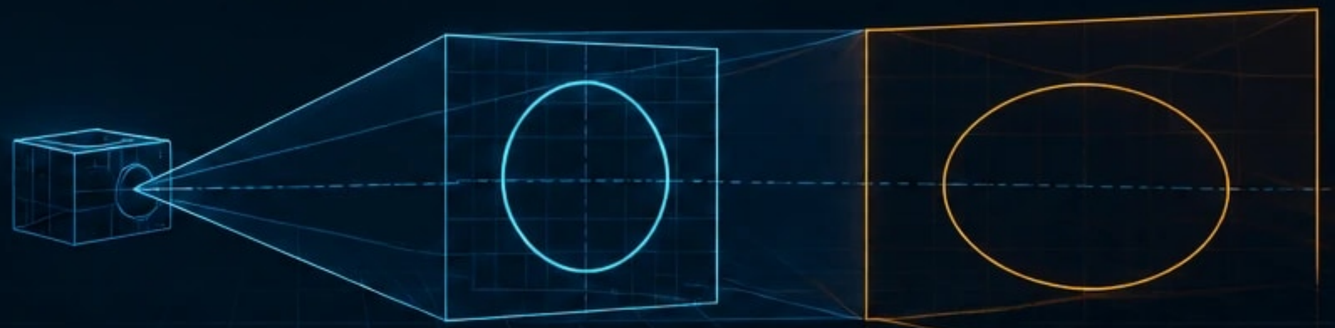
Circle ↓ **Ellipse**

Karena skala horizontal/ vertical tidak sesuai.

Pada WebGL:

```
const aspect =
 canvas.width / canvas.height;
```





Selalu sesuaikan aspect projection dengan ukuran **viewport**.

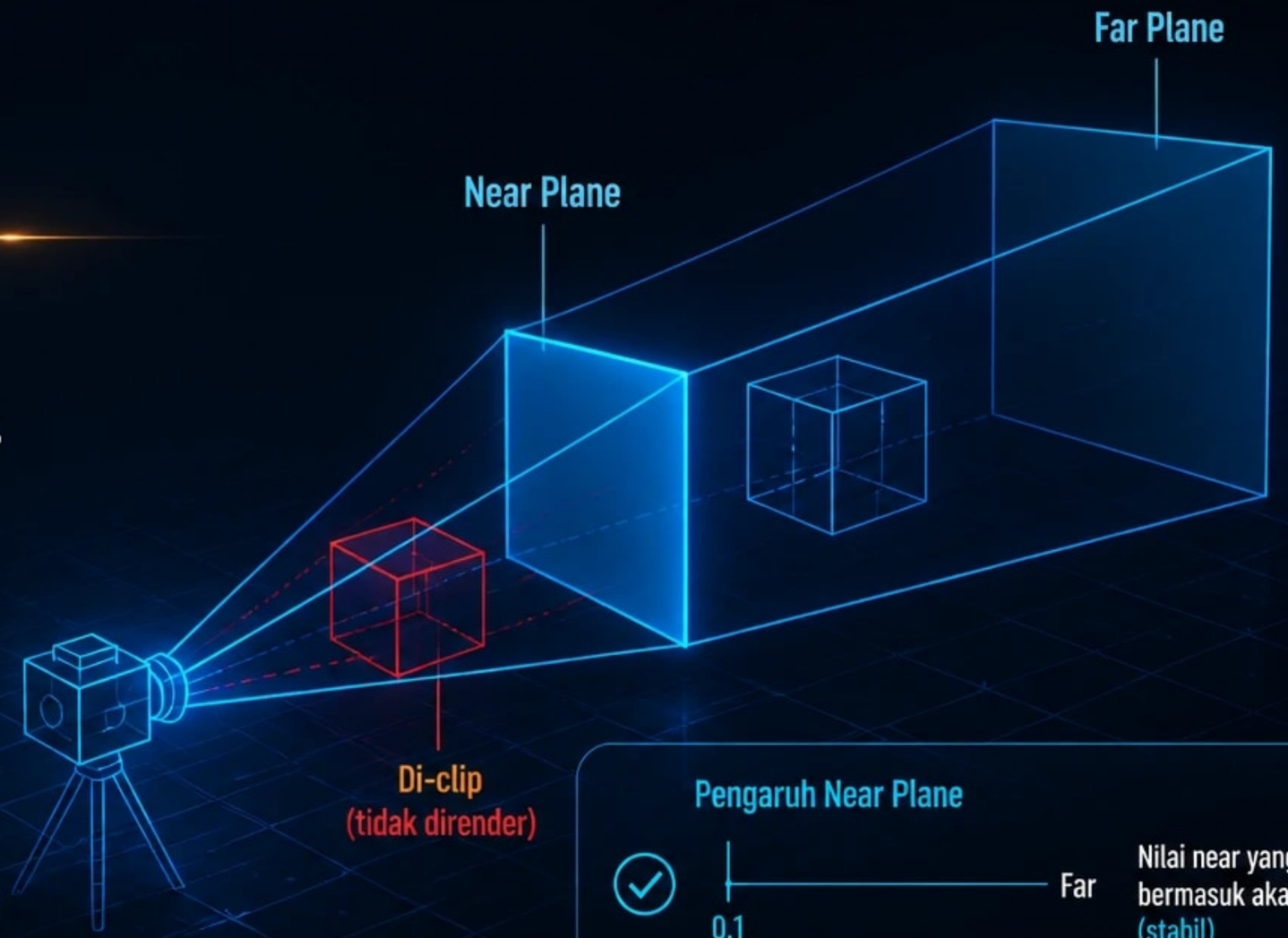
Near Plane

Near Plane adalah **batas terdekat camera** yang dirender.

Object lebih dekat dari near plane:

di-clip

Near plane tidak sebaiknya **terlalu dekat ke nol** tanpa alasan.



Pengaruh Near Plane

0.1 — Far
Nilai near yang bermasuk akal (stabil)

0.0001 — Far
Terlalu dekat ke nol (presisi depth buruk, flicker, artifacts)

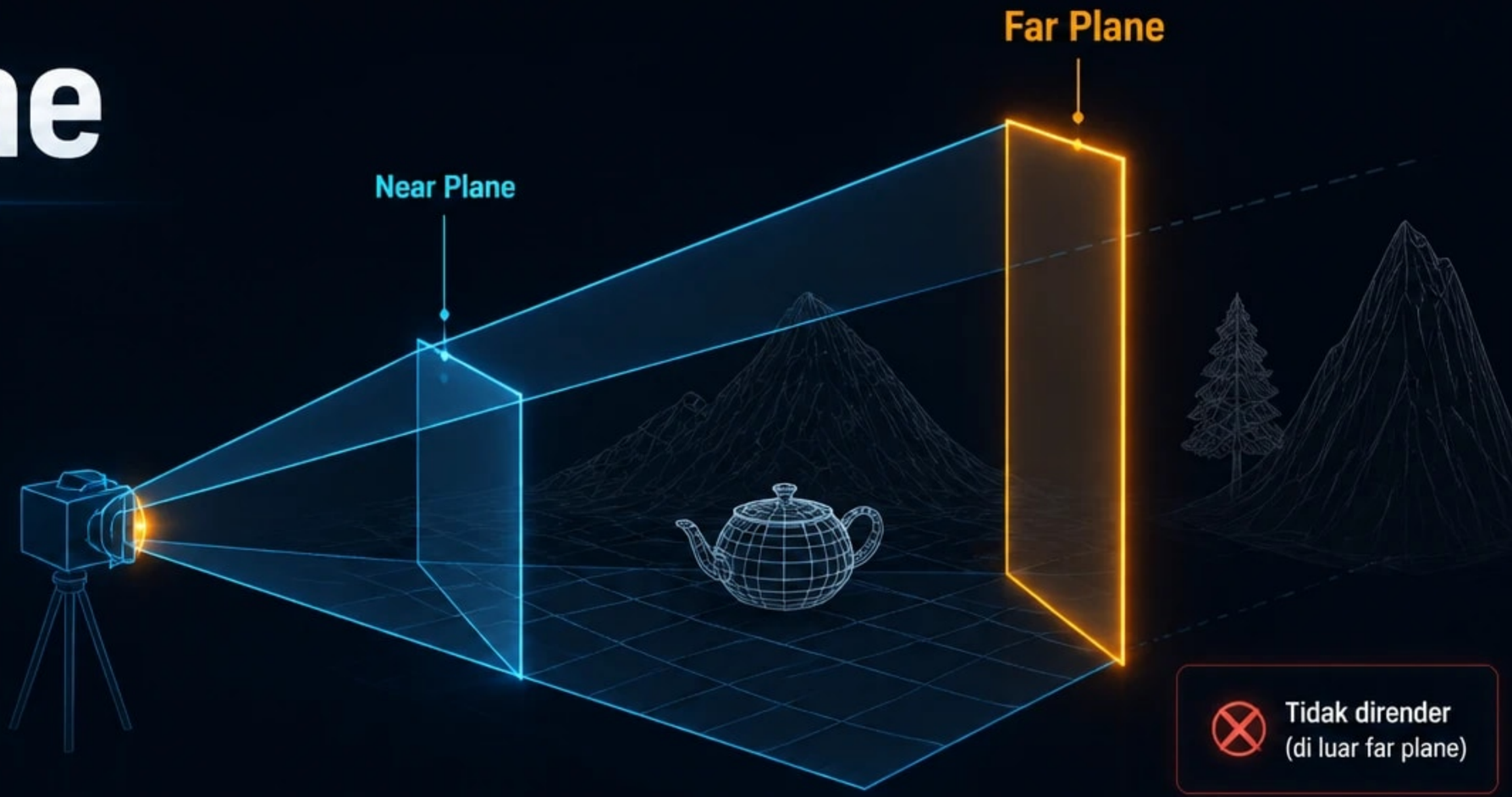
Far Plane

Far Plane adalah batas terjauh yang dirender.

Object lebih jauh dari far plane:

tidak terlihat

Visible scene berada antara **near** dan **far.**





Near/Far dan Depth Precision

Rentang ekstrem:

```
near = 0.001
far  = 1,000,000
```

dapat menurunkan **presisi depth.**

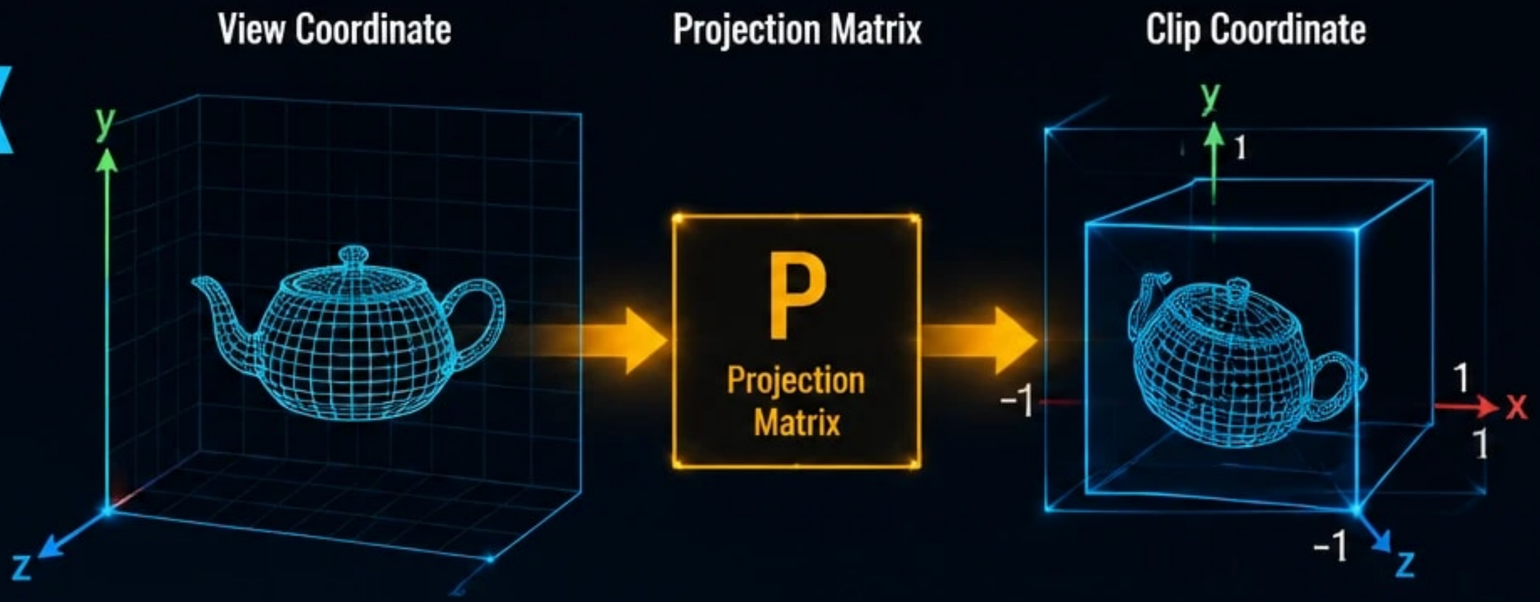
Prinsip praktis:

- **near** jangan terlalu kecil,
- **far** jangan terlalu jauh bila tidak perlu.





Projection Matrix



Projection Matrix mengubah:

- View Coordinate
- Clip Coordinate

Mengubah posisi dari ruang pandang (view space) menjadi ruang ciip (clip space).

Jenis:

- Orthographic Matrix
- Perspective Matrix

Orthographic Matrix



Parallel projection (tanpa perspektif)

Perspective Matrix



Perspective projection (dengan perspektif)

Clip Space dan Perspective Divide

Setelah Projection Matrix:

Clip Position (x, y, z, w)
Hasil transformasi oleh Projection Matrix

↓

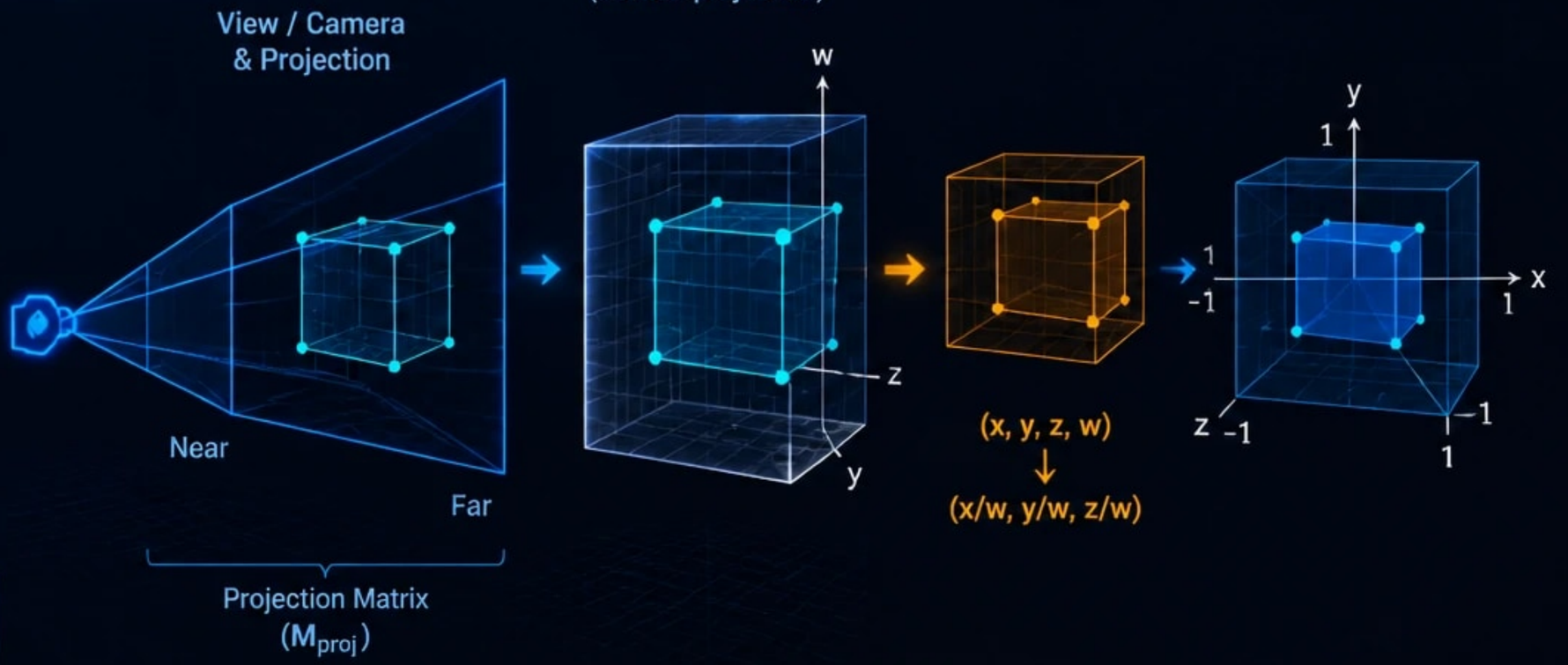
Perspective Divide
Bagi setiap komponen dengan w

↓

(x/w, y/w, z/w)
Koordinat setelah perspektif

↓

NDC (Normalized Device Coordinate)
Rentang: x, y, z ∈ [−1, 1]

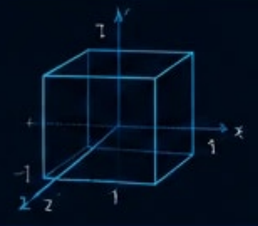


NDC telah diperkenalkan pada Pertemuan 2.

Viewport Transform

NDC

Normalized Device Coordinate

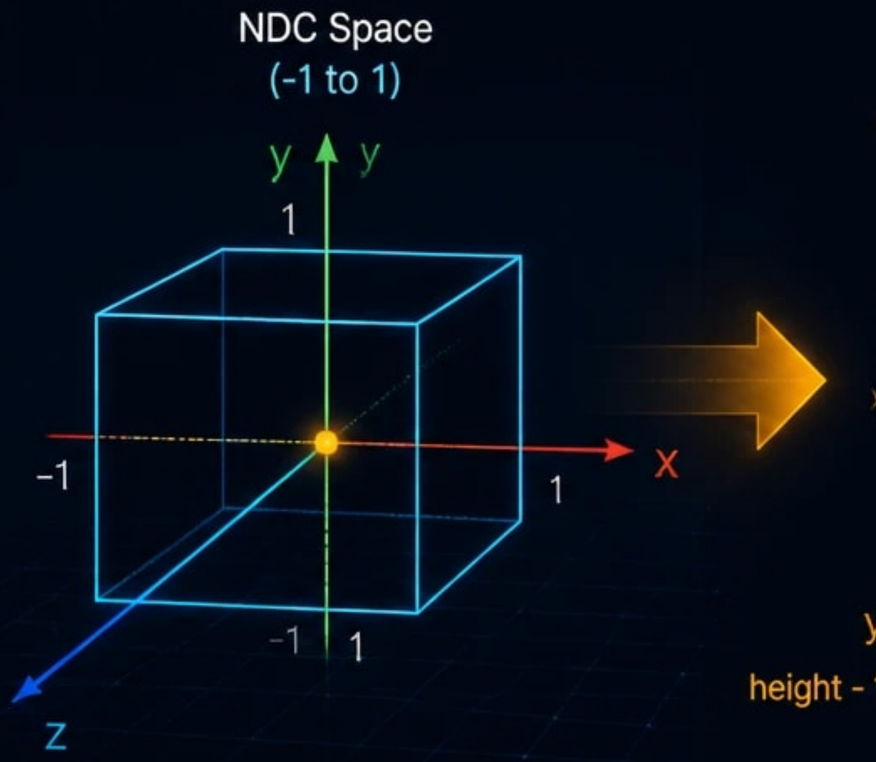


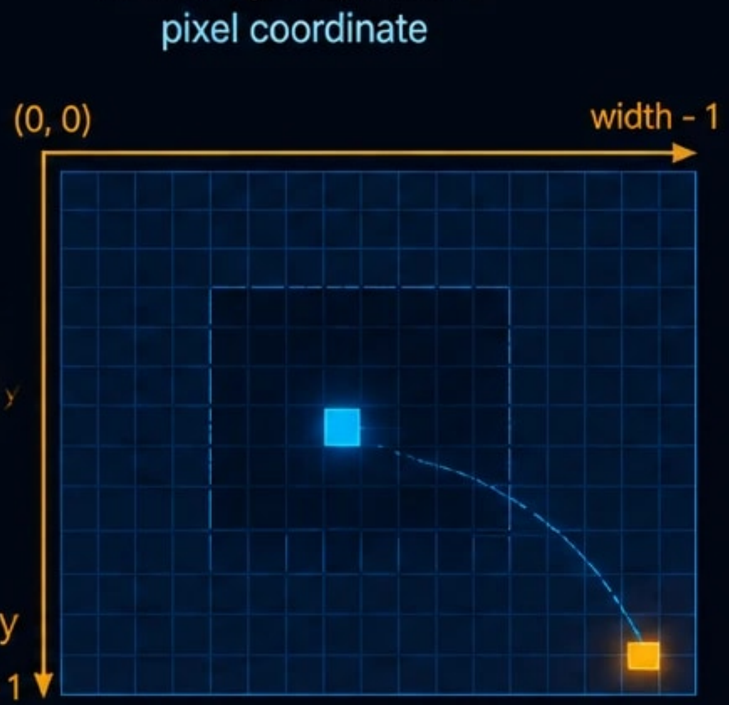
Viewport Transform

Mapping dari NDC ke ukuran Canvas (pixel)

Screen Coordinate

Koordinat layar menggunakan pixel sesuai ukuran Canvas





Hasil akhir berada pada **framebuffer (Canvas)** dalam satuan **pixel (x, y)**.

Viewport Transform memetakan **NDC [-1, 1]** ke koordinat **pixel** pada **Canvas**.

NDC (-1,1) → Viewport Transform → Screen (pixel)

MODEL VIEW PROJECTION

TIGA MATRIX UTAMA

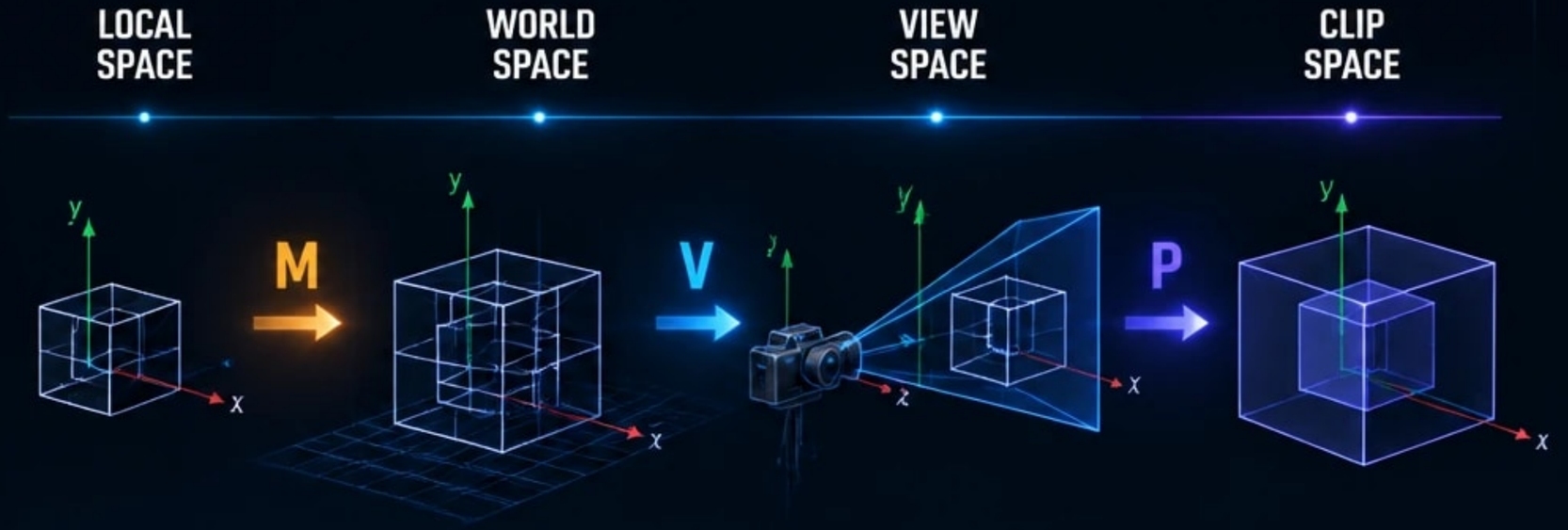
M = Model Matrix

V = View Matrix

P = Projection Matrix

PIPELINE TRANSFORMASI

Local Space
↓ M Model Matrix
World Space
↓ V View Matrix
View Space
↓ P Prew Space
Clip Space
↓ P Projection Matrix

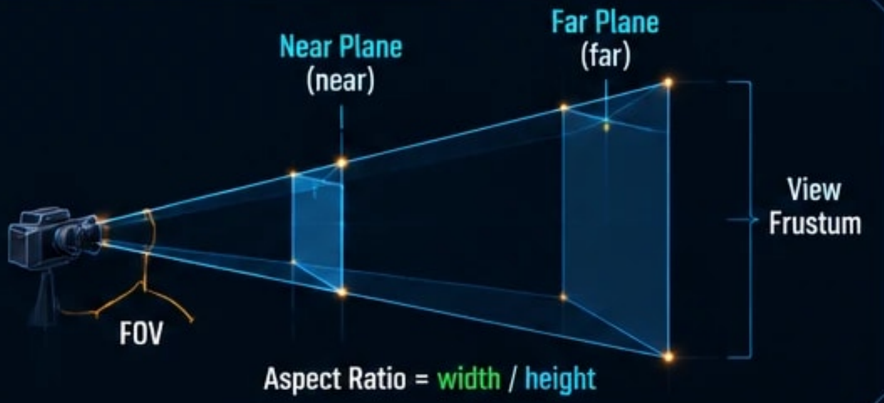


IDE UTAMA

Posisi vertex diubah berurutan:

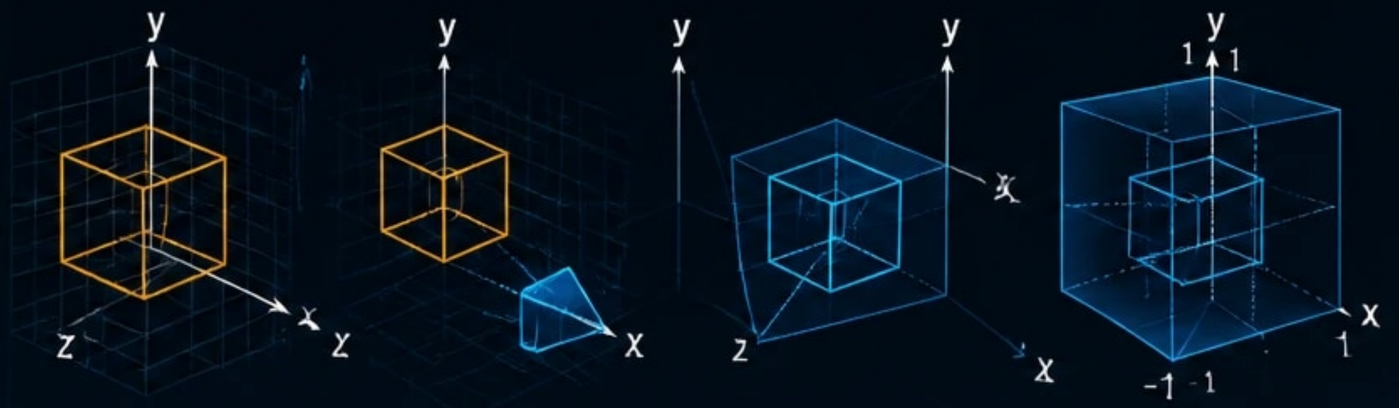
$$clipPos = P \times V \times M \times localPos$$

Homogeneous Coordinate (x, y, z, w)



MVP Matrix

LOCAL SPACE → **M** Model Matrix → **V** View Matrix → **P** Projection Matrix → CLIP SPACE



Secara konseptual:

$$\text{clipPosition} = \underbrace{P}_{\text{Projection}} \times \underbrace{V}_{\text{View}} \times \underbrace{M}_{\text{Model}} \times \text{localPosition}$$

- **M (Model)** : transformasi objek dari local space ke world space
- **V (View)** : transformasi dari world space ke view/eye space
- **P (Projection)**: transformasi dari view space ke clip space

Urutan mengikuti convention matrix yang digunakan.

Ini mengabungkan hasil P3 dan P4.

$$MVP = P \times V \times M$$

$$\underbrace{\begin{bmatrix} p_{00} & p_{01} & p_{02} & p_{03} \\ p_{10} & p_{11} & p_{12} & p_{13} \\ p_{20} & p_{21} & p_{22} & p_{23} \\ p_{30} & p_{31} & p_{32} & p_{33} \end{bmatrix}}_{P} \times \underbrace{\begin{bmatrix} v_{00} & v_{01} & v_{02} & v_{03} \\ v_{10} & v_{11} & v_{12} & v_{13} \\ v_{20} & v_{21} & v_{22} & v_{23} \\ v_{30} & v_{31} & v_{32} & v_{33} \end{bmatrix}}_{V} \times \underbrace{\begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ m_{30} & m_{31} & m_{32} & m_{33} \end{bmatrix}}_{M}$$

MVP Matrix membawa posisi vertex dari **local** space → **clip space**.

MVP pada Vertex Shader

Vertex Shader sudah dipelajari pada P2.
Sekarang kita hanya menggunakan:

```
gl_Position =
  u_projection *
  u_view *
  u_model *
    vec4(a_position, 1.0);
```

Fokus slide ini:

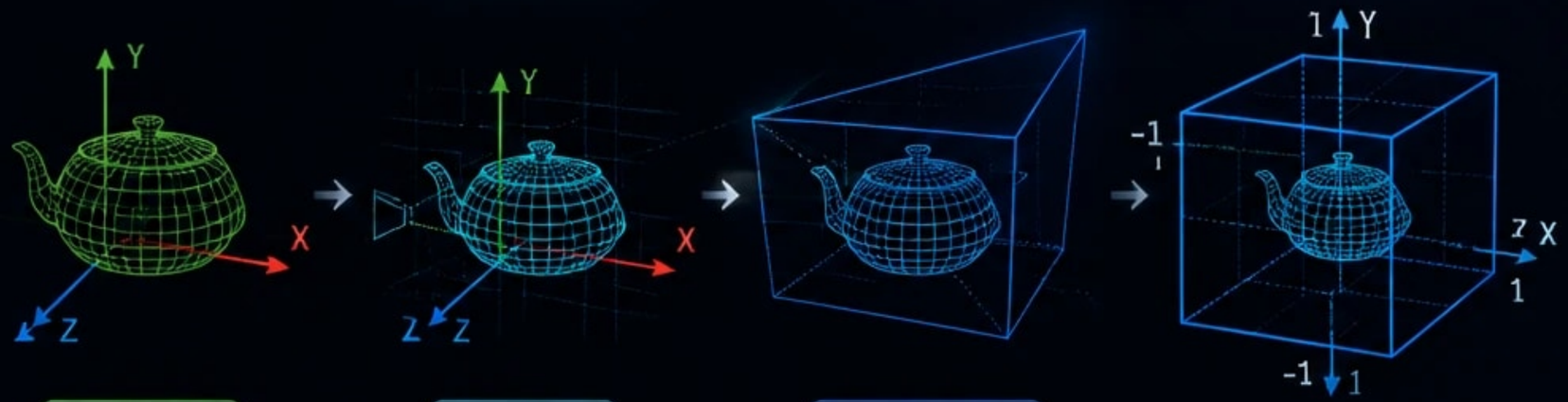
hubungan Model, View, dan Projection.

MODEL
Object Space

VIEW
Eye (View) Space

PROJECTION
Clip Space

CLIP SPACE
NDC (-1 .. 1)



u_model → u_view → u_projection

```
gl_Position = u_projection * u_view * u_model * vec4(a_position, 1.0)
```

Model
Menempatkan objek di dunia

→ **View**
Menempatkan dunia terhadap kamera

→ **Projection**
Memproyeksikan ke ruang clip

→ **NDC → Screen**
Siap untuk rasterization

Dari 2D ke 3D

Pada P2:

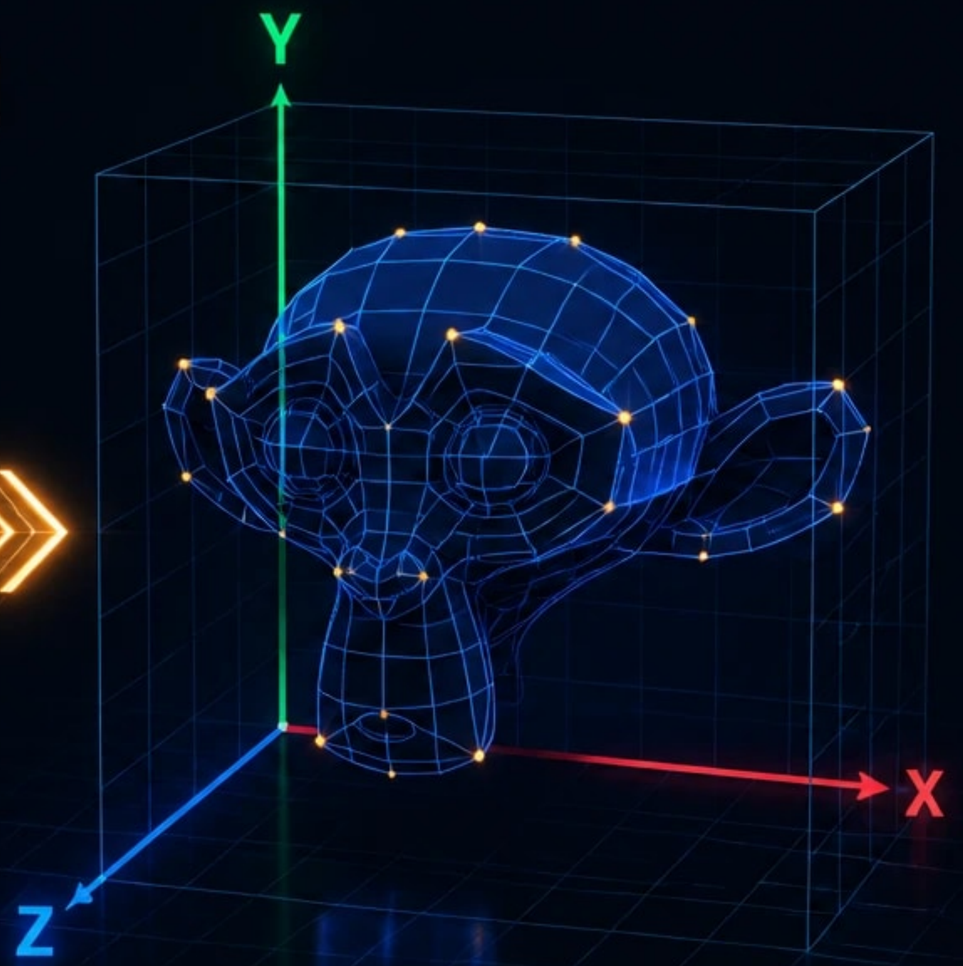
```
in vec2 a_position;
```

Untuk 3D:

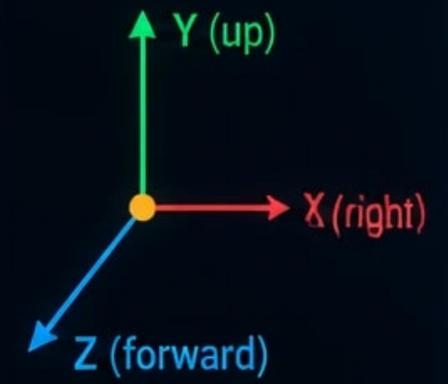
```
in vec3 a_position;
```

Setiap vertex memiliki:

X, Y, Z



Koordinat 3D



X → kiri / kanan

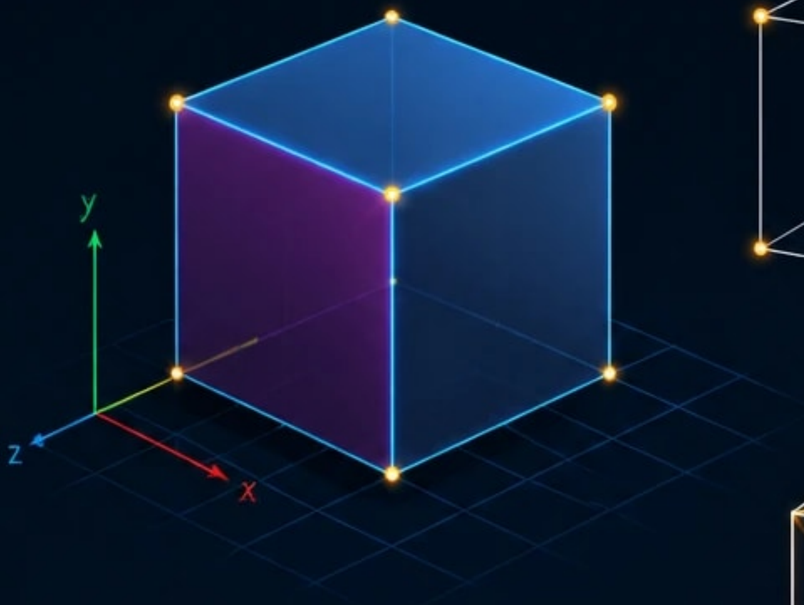
Y → atas / bawah

Z → depan / belakang

Cube sebagai Object 3D

Cube mempunyai:

- **8** posisi sudut secara geometris,
- **6** faces,
- **12** triangles.



Untuk rendering per-face attribute, **vertex** dapat diduplikasi.

Cube ideal sebagai objek latihan:

- transform
- camera
- projection
- depth



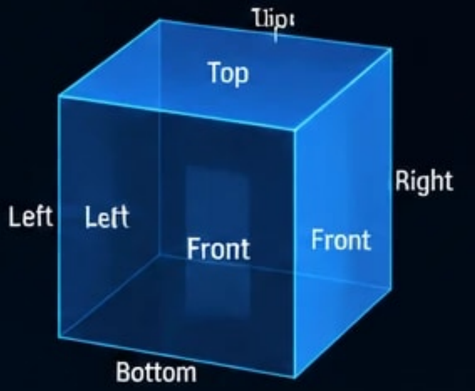
Vertex Duplication (per-face attribute)



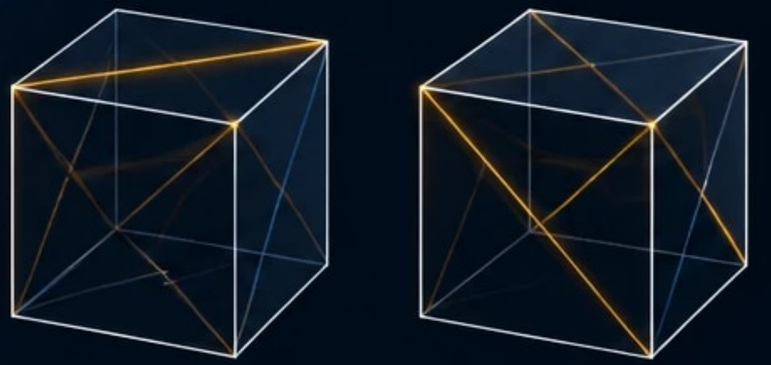
8 Vertices



6 Faces



12 Triangles (2 per face)





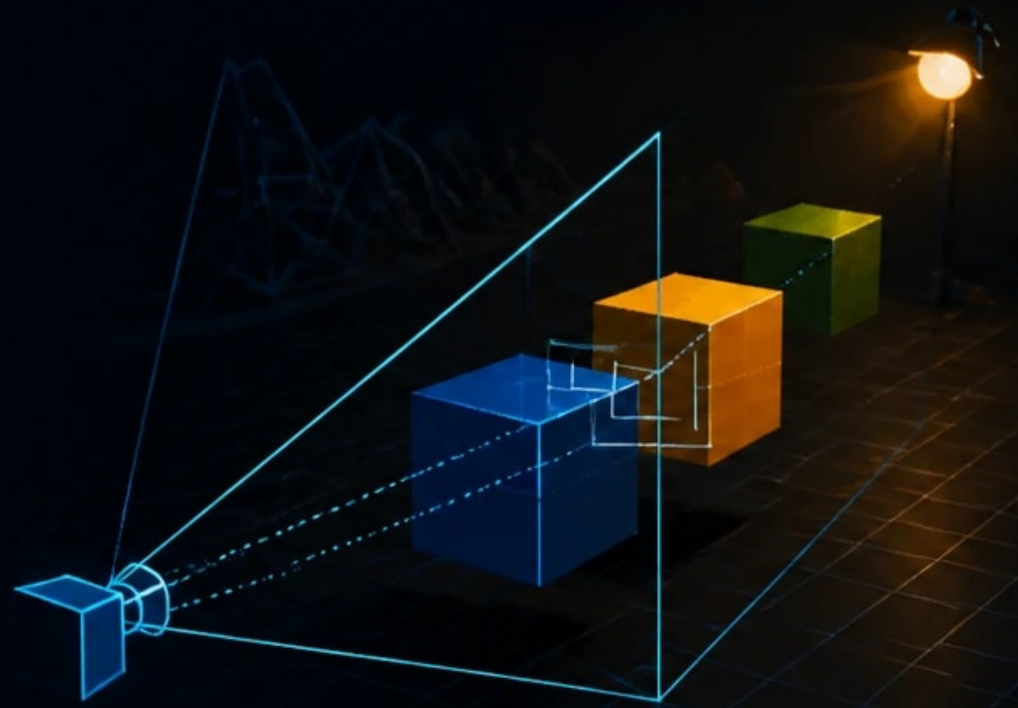
Mengapa Depth Dibutuhkan?

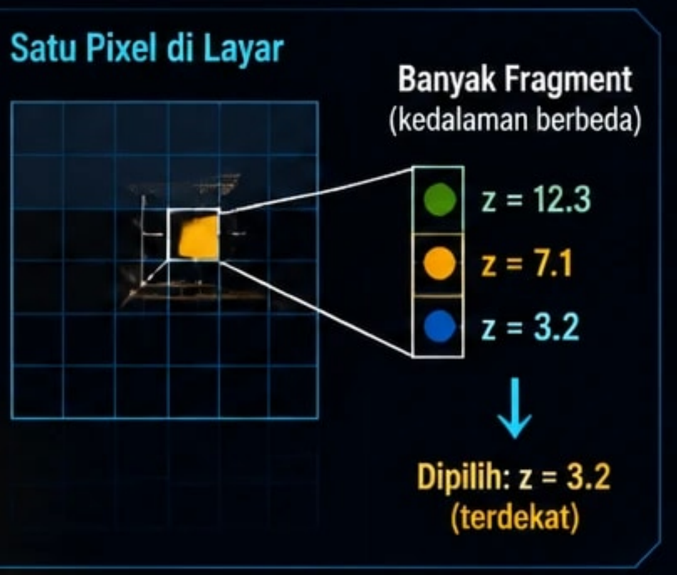
Pada scene 3D, beberapa fragment dapat berada pada pixel layar yang sama.

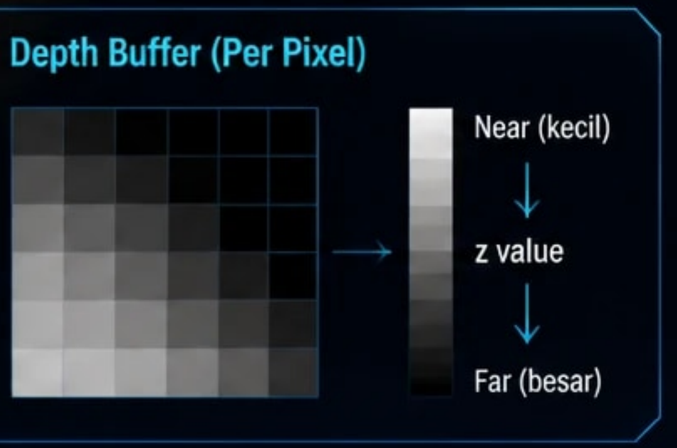
Kita perlu menentukan: fragment mana yang paling dekat camera?

Solusi:

Depth Buffer







Depth Buffer

Depth Buffer menyimpan **informasi kedalaman.**

- Untuk setiap fragment:

New Depth

↓

Compare

↓

Keep / Reject

Depth buffer bekerja bersama **color buffer.**





Depth test memastikan hanya **fragment terdekat** yang menulis ke **color buffer.**

Depth Test

Aktifkan Depth Test

```
gl.enable(gl.DEPTH_TEST);
```

Bersihkan sebelum render frame

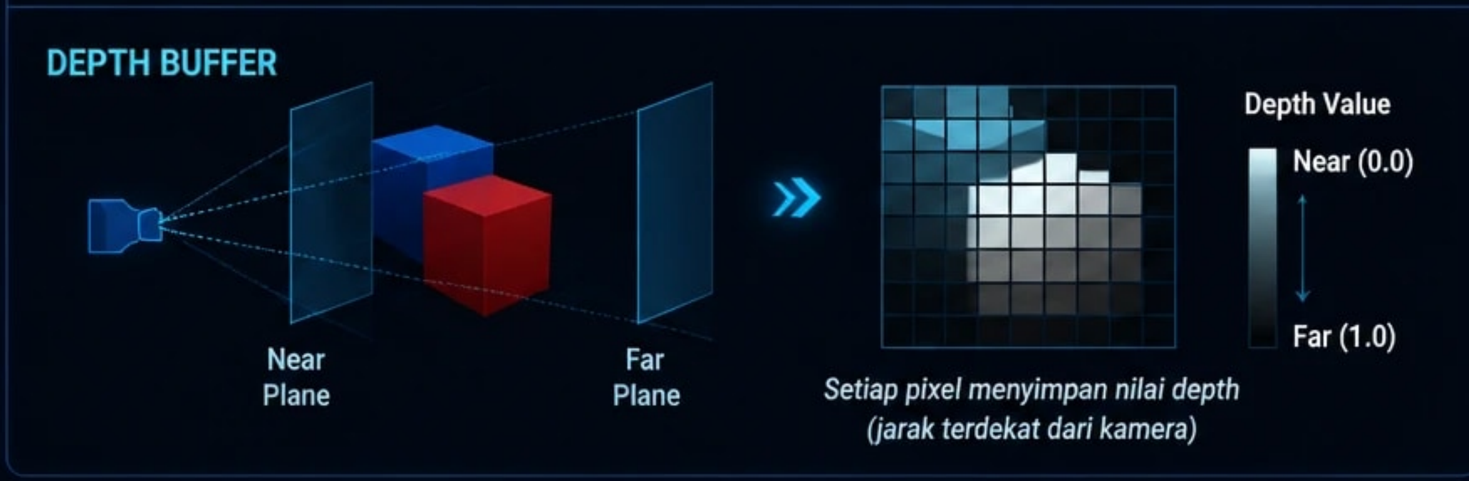
```
gl.clear(
    gl.COLOR_BUFFER_BIT |
    gl.DEPTH_BUFFER_BIT
);
```

Tanpa depth test,
urutan draw dapat menghasilkan visual 3D yang salah.

Pertanyaan: Saya mempunyai objek segitiga dan persegi, objek segitiga berada pada kedalaman -0.75 dan objek B berada pada ke dalaman -0.5.

Kedua objek tersebut beririsan. Bagaimana tampilan di layar jika

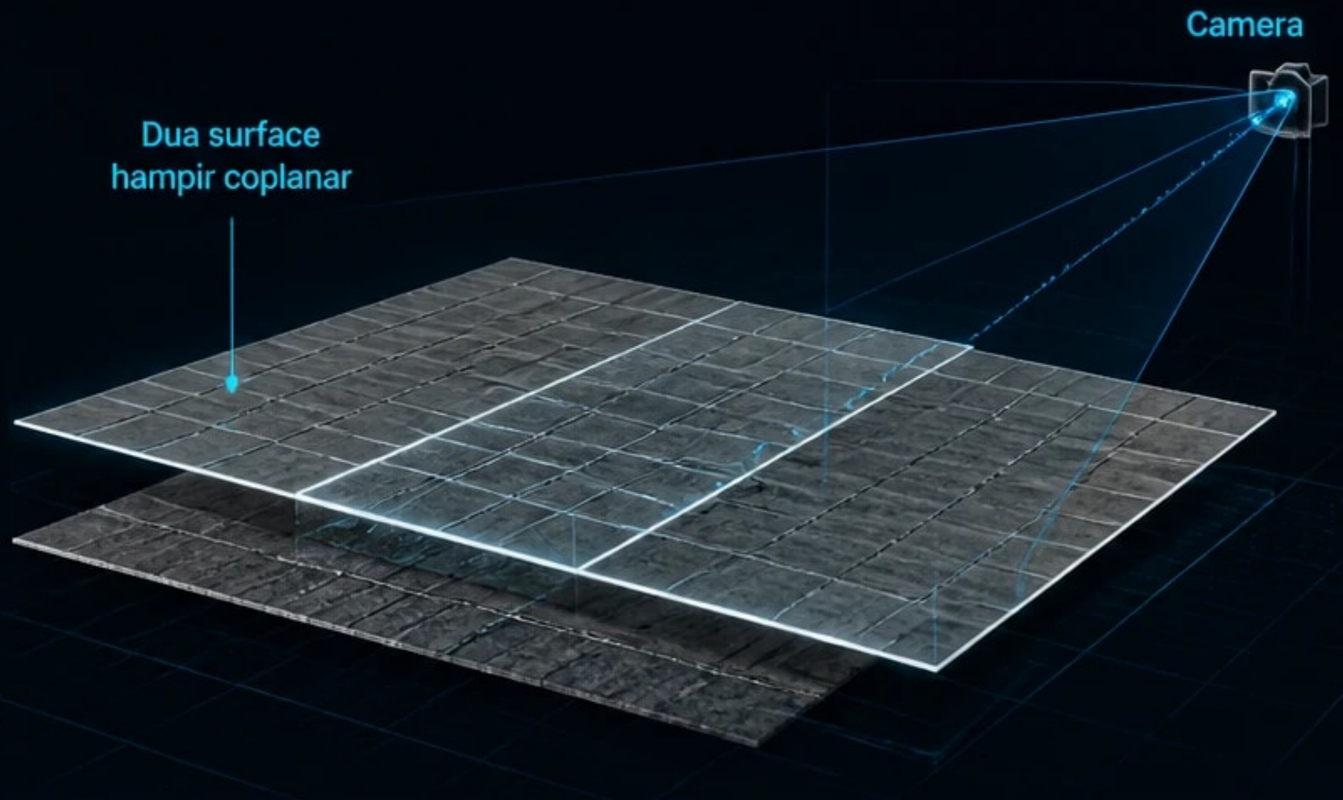
objek B di gambar terlebih dahulu dan tidak menggunakan depth buffer?



Grafika Komputer

Z-Fighting

Z-fighting terjadi ketika dua surface memiliki **depth yang hampir sama**.



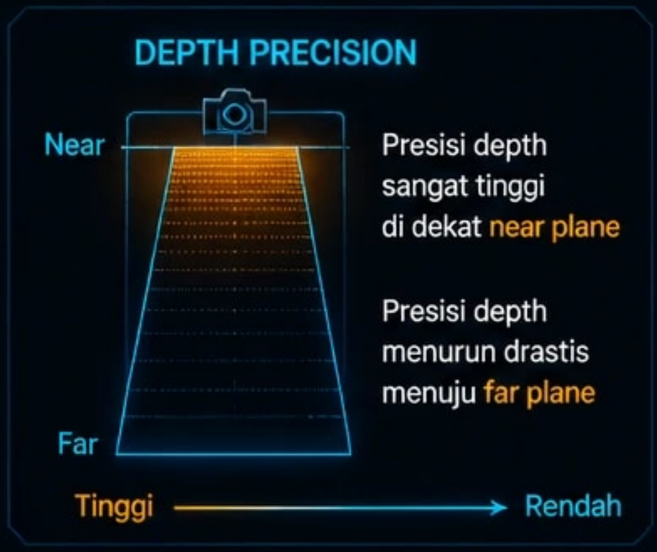
GEJALA

- flickering
- pola saling menimpa

FAKTOR

- surface hampir coplanar
- near terlalu dekat
- far terlalu jauh
- depth precision terbatas



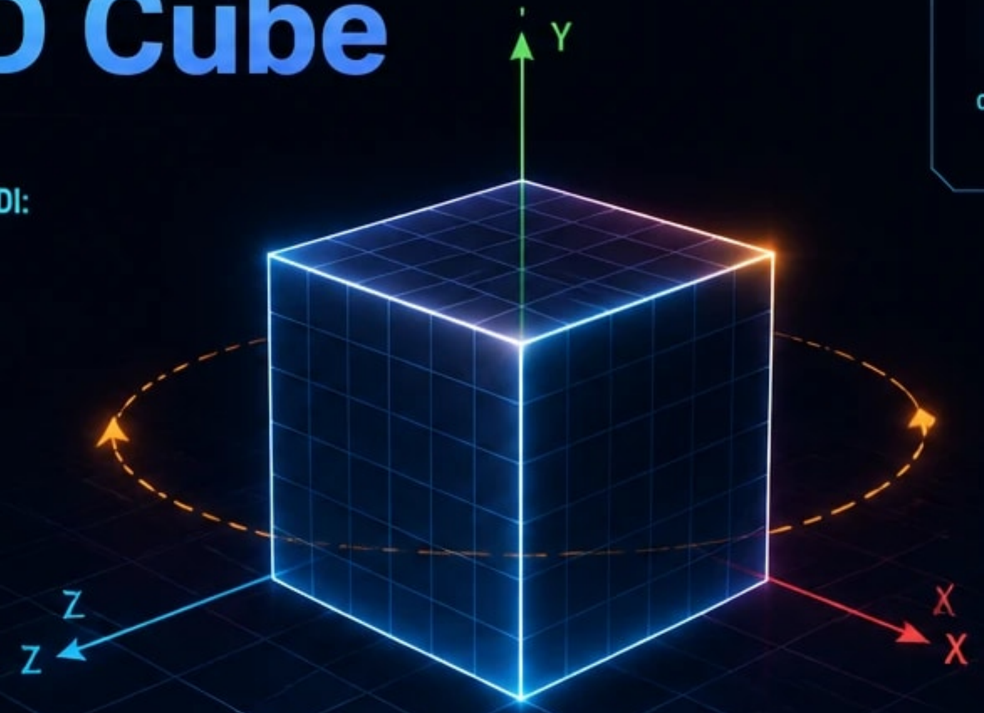


Grafika Komputer

Praktikum: Rotating 3D Cube

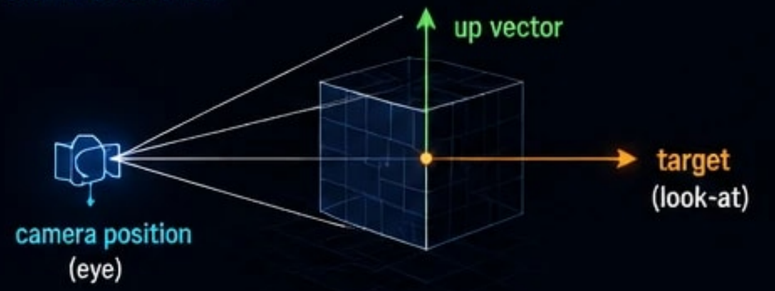
MAHASISWA MENGEMBANGKAN OBJECT MENJADI:

- cube 3D
- Model Matrix
- camera position
- target
- up vector
- View Matrix
- Perspective Projection
- Orthographic Projection
- depth test
- rotation animation

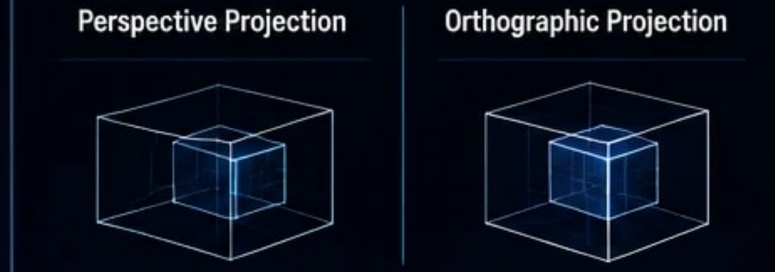


ROTATION ANIMATION

CAMERA & VIEW

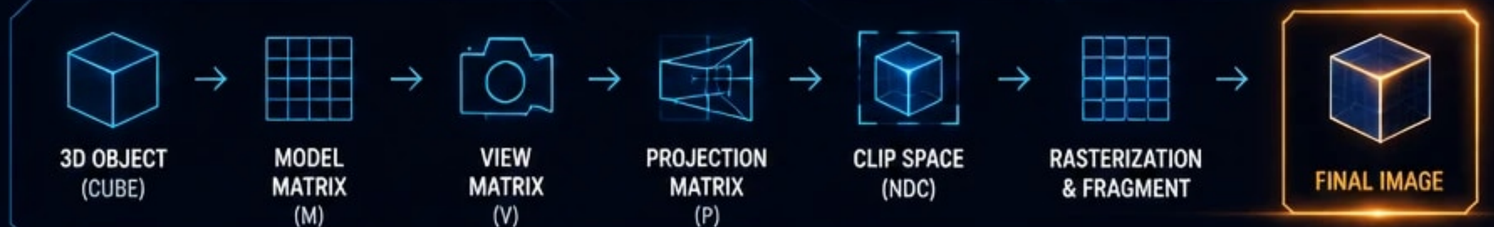


PROJEKSI

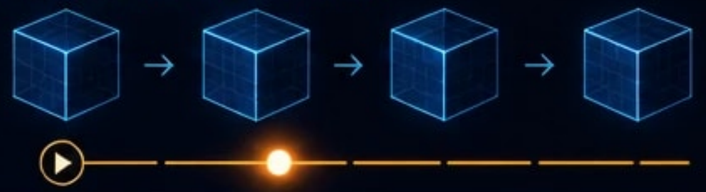


DEPTH TEST



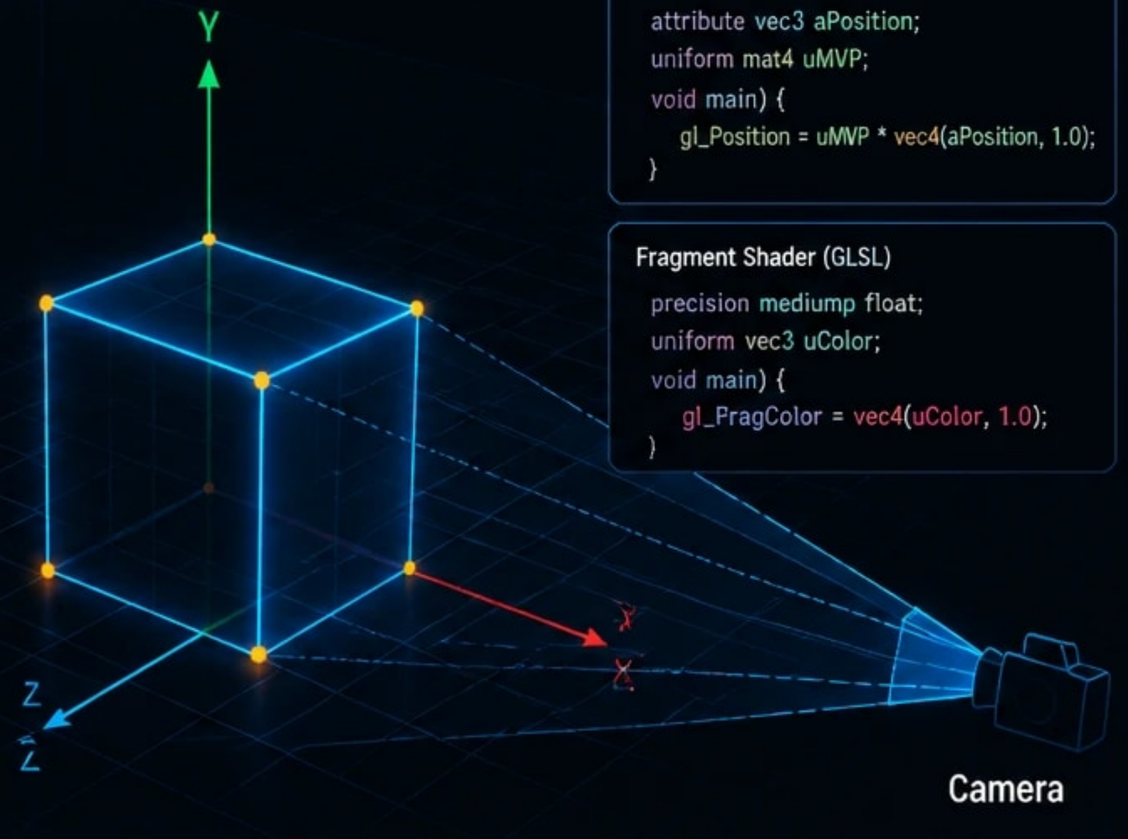


ROTATION ANIMATION (Y-AXIS)



Rencana Praktikum

1. Buat Cube Geometry
2. Gunakan vec3 Position
3. Gunakan Model Matrix P3
4. Buat View Matrix
5. Buat Perspective Matrix
6. Buat Orthographic Matrix
7. Gabungkan MVP
8. Aktifkan Depth Test
9. Tambahkan Rotation
10. Tambahkan Camera Control



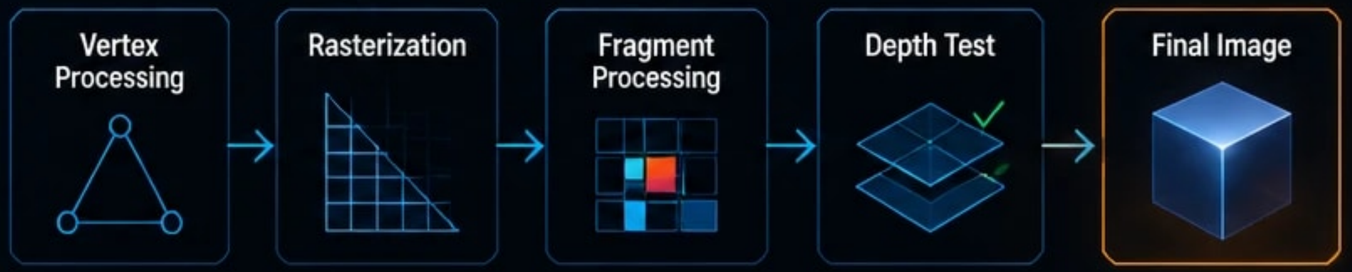
Vertex Shader (GLSL)

```
attribute vec3 aPosition;
uniform mat4 uMVP;
void main) {
  gl_Position = uMVP * vec4(aPosition, 1.0);
}
```

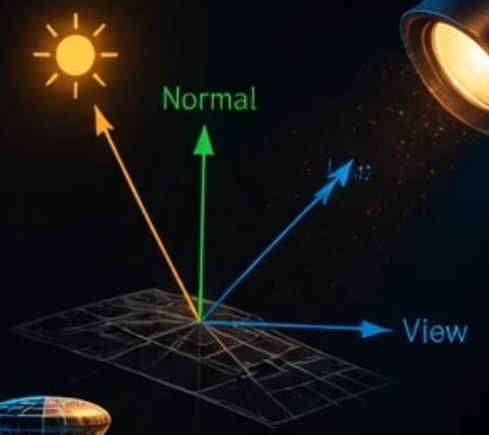
Fragment Shader (GLSL)

```
precision mediump float;
uniform vec3 uColor;
void main) {
  gl_FragColor = vec4(uColor, 1.0);
}
```

Detail teknis ada di **modul praktikum.**



TERIMA KASIH



Pertemuan Berikutnya

Lighting, Shading & Texture pada WebGL



